

主要内容

- 图灵机简介、形式化定义
- 图灵机的表示能力：语言、停机与整数运算
- 扩展图灵机
- 受限图灵机

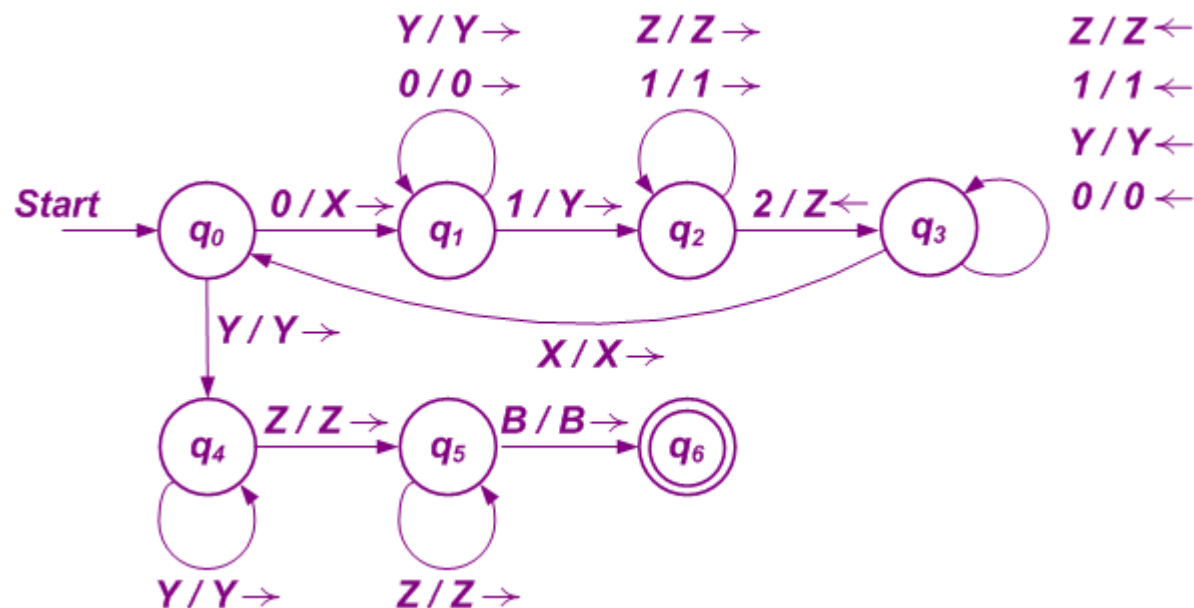
图灵机的概念与定义

◇ 课程回顾 (Lecture 1)

设 $\Sigma = \{0, 1, 2\}$, $L = \{0^n 1^n 2^n \mid n \geq 1\}$.

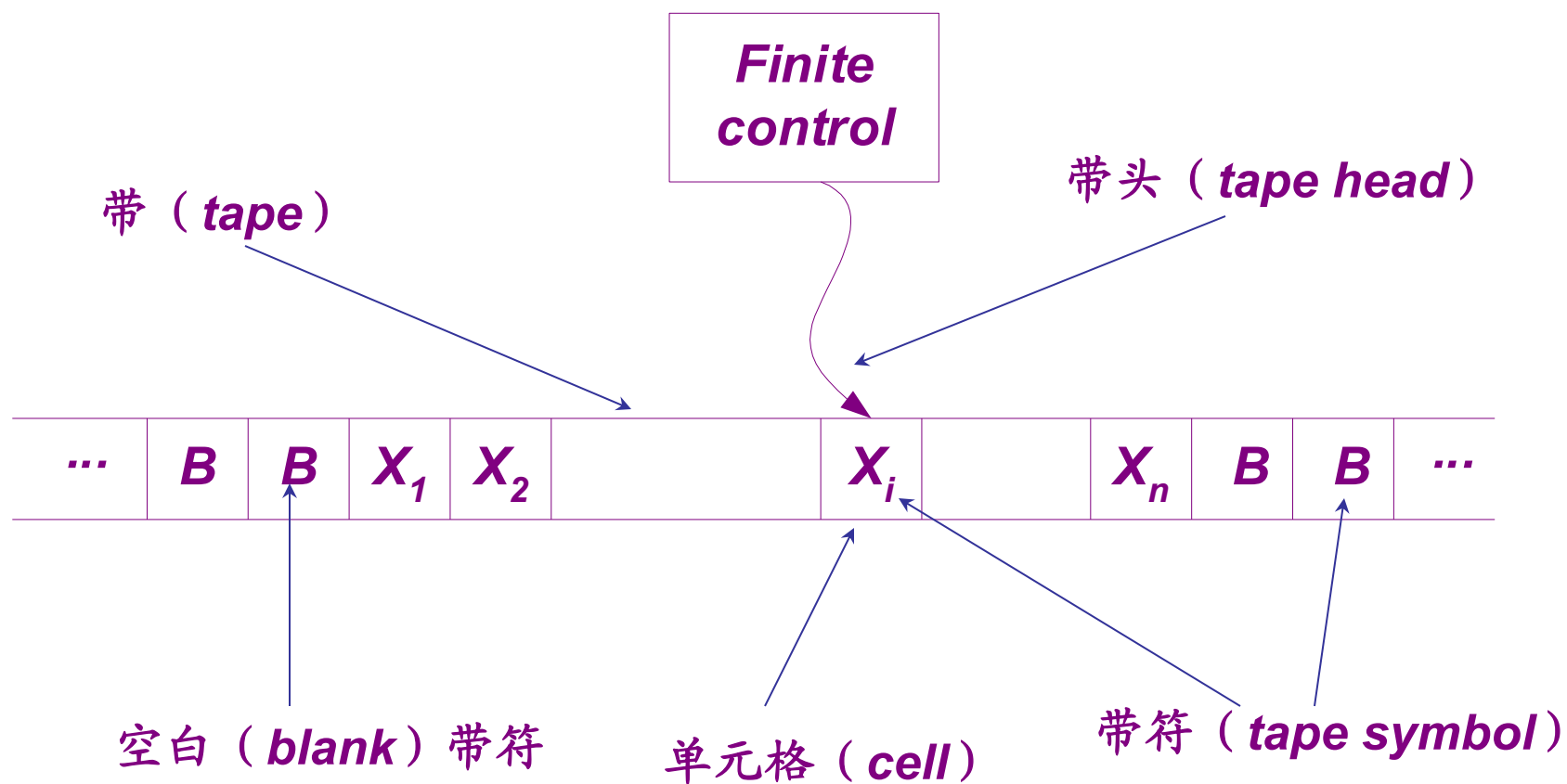
无有穷自动机和上下文无关文法能够接受语言 L .

存在一个图灵机可接受该语言.



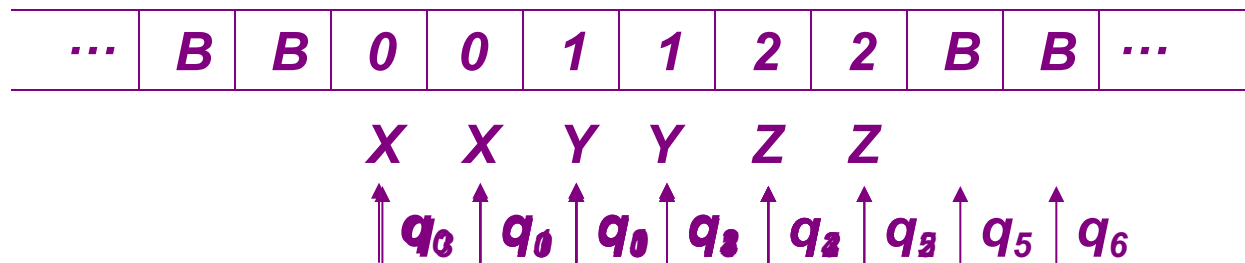
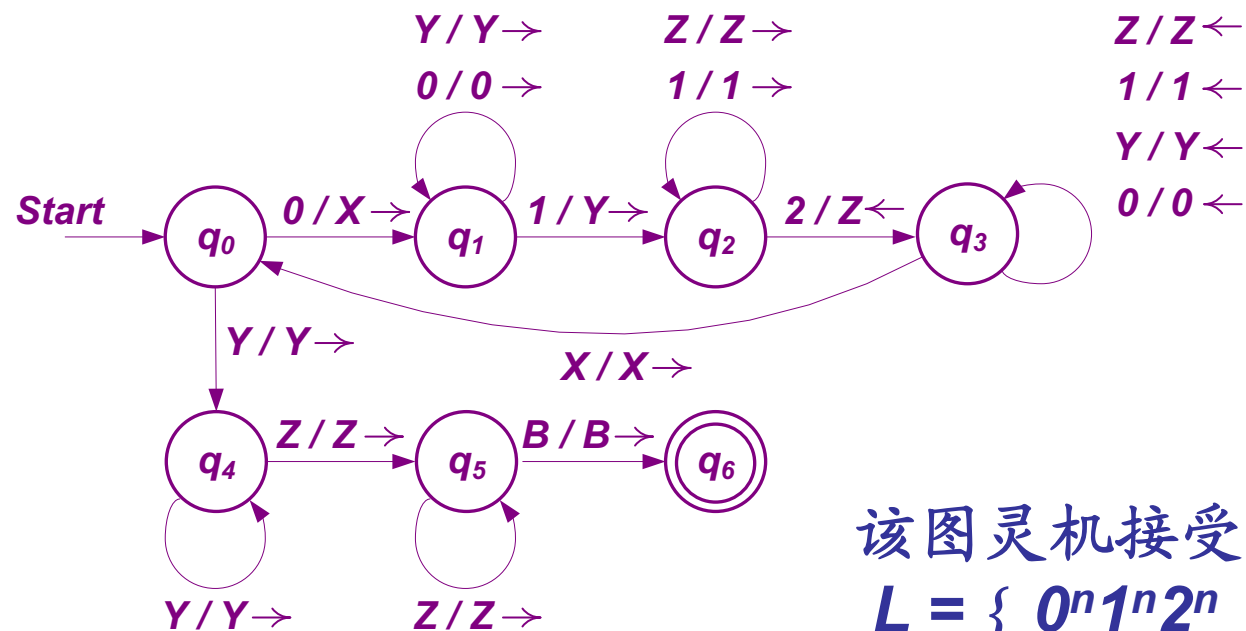
图灵机的概念与定义

◇ 基本图灵机



图灵机的概念与定义

✧ 举例



图灵机的概念与定义

◇ 形式定义 一个图灵机 TM (Turing machine) 是一个七元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F).$$

有穷状态集

有穷输入符号集

有穷带符号集

转移函数

开始状态

特殊带符：空白符

终态集合

$$q_0 \in Q$$

$$\Sigma \subseteq \Gamma$$

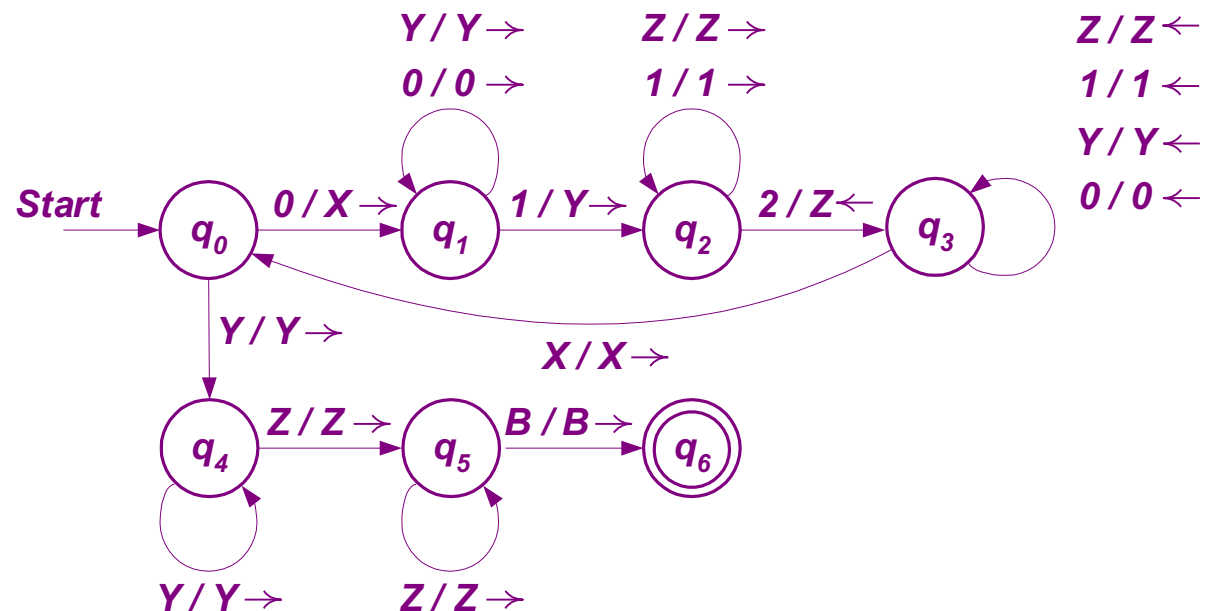
$$B \in \Gamma - \Sigma$$

$$F \subseteq Q$$

转移函数为偏函数 $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$

图灵机的概念与定义

◇ 举例（续前例）



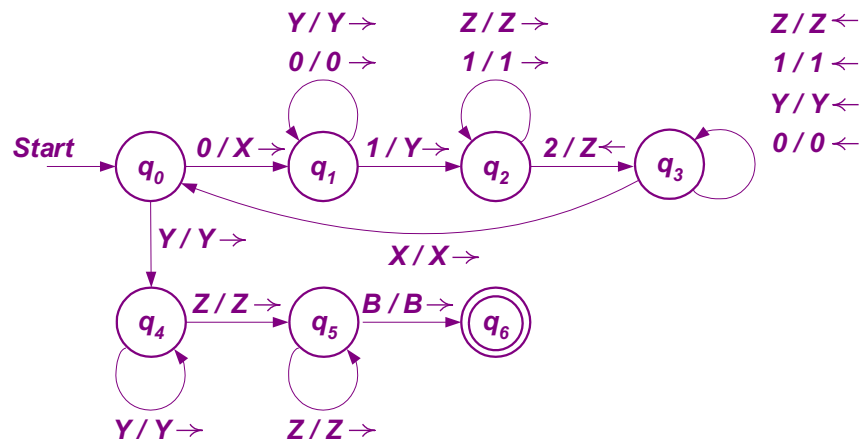
该图灵机的七元组形式为：

$$M = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}, \{0, 1, 2\}, \{0, 1, 2, X, Y, Z, B\}, \delta, q_0, B, \{q_6\}).$$

转移函数可由上图的转移图形式给出。

图灵机的概念与定义

◇ 转移图与转移表



State	Symbol						
	0	1	2	X	Y	Z	B
$\rightarrow q_0$	(q_1, X, R)	—	—	—	(q_4, Y, R)	—	—
q_1	$(q_1, 0, R)$	(q_2, Y, R)	—	—	(q_1, Y, R)	—	—
q_2	—	$(q_2, 1, R)$	(q_3, Z, L)	—	—	(q_2, Z, R)	—
q_3	$(q_3, 0, L)$	$(q_3, 1, L)$	—	(q_0, X, R)	(q_3, Y, L)	(q_3, Z, L)	—
q_4	—	—	—	—	(q_4, Y, R)	(q_5, Z, R)	—
q_5	—	—	—	—	—	(q_5, Z, R)	(q_6, B, R)
$* q_6$	—	—	—	—	—	—	—

图灵机的概念与定义

✧ 用 ID (*instantaneous descriptions*) 表达瞬时描述

图灵机 $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ 的瞬时描述用字符串 $X_1X_2\cdots X_{i-1}qX_iX_{i+1}\cdots X_n$ 表示, 称为 ID , 其中

1. $q \in Q$ 为当前 M 的状态;
2. 当前带的头正在扫描 X_i ;
3. $X_1X_2\cdots X_n$ 为当前带中最左端的非空白符号与最右端的非空白符号之间的带符号串. 有两个例外, 即当带头位于最左端非空白符号的左边时, $X_1X_2\cdots X_n$ 的某个前缀部分为空白符号串; 当带头位于最右端非空白符号的右边时, $X_1X_2\cdots X_n$ 的某个后缀部分为空白符号串.

图灵机的概念与定义

◇ 给定图灵机 $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ ，定义 ID's 之间的转移关系 $\vdash_M$ 如下：

1. 设 $\delta(q, X_i) = (p, Y, L)$ ，则有

$$X_1 X_2 \dots X_{i-1} q X_i X_{i+1} \dots X_n \vdash_M X_1 X_2 \dots X_{i-2} p X_{i-1} Y X_{i+1} \dots X_n,$$

但有如下两个例外：

(1) $i=1$ 时， $q X_1 X_2 \dots X_n \vdash_M p B Y X_2 \dots X_n$ ，和

(2) $i=n$ 及 $Y=B$ 时， $X_1 X_2 \dots X_{n-1} q X_n \vdash_M X_1 X_2 \dots X_{n-2} p X_{n-1}$ 。

2. 设 $\delta(q, X_i) = (p, Y, R)$ ，则有

$$X_1 X_2 \dots X_{i-1} q X_i X_{i+1} \dots X_n \vdash_M X_1 X_2 \dots X_{i-1} Y p X_{i+1} \dots X_n,$$

但有如下两个例外：

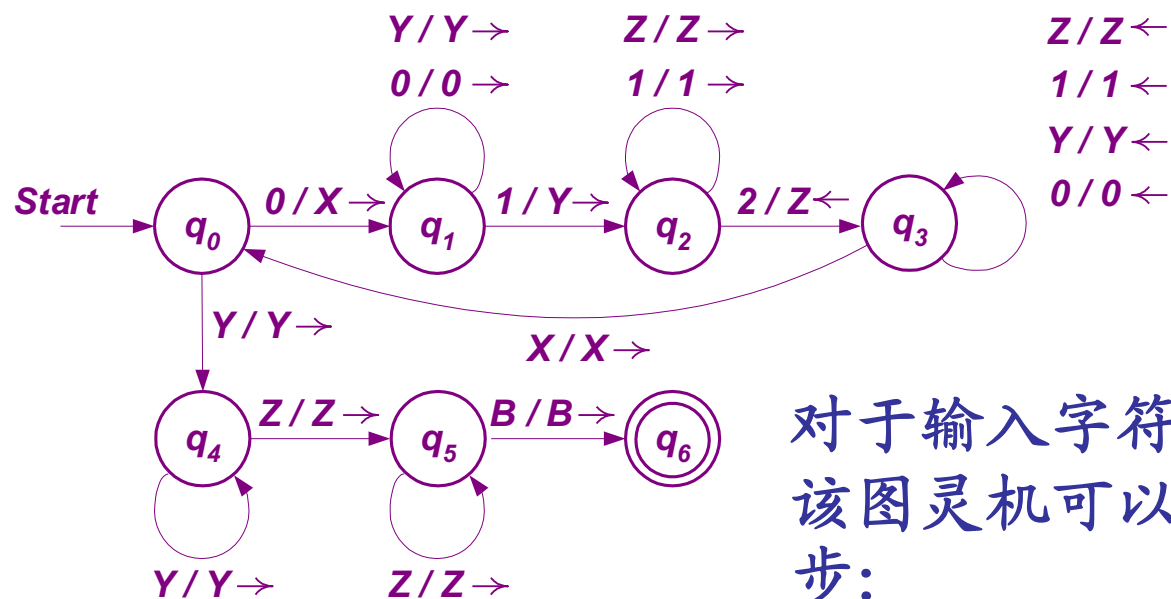
(1) $i=n$ 时， $X_1 X_2 \dots X_{n-1} q X_n \vdash_M X_1 X_2 \dots X_{n-1} Y p B$ ，和

(2) $i=1$ 及 $Y=B$ 时， $q X_1 X_2 \dots X_n \vdash_M p X_2 \dots X_{n-1} X_n$ 。

◇ 上述关系 $\vdash_M$ 的零步或多步转移记为 $\vdash_M^*$ (或 $\vdash^*$)。

图灵机的概念与定义

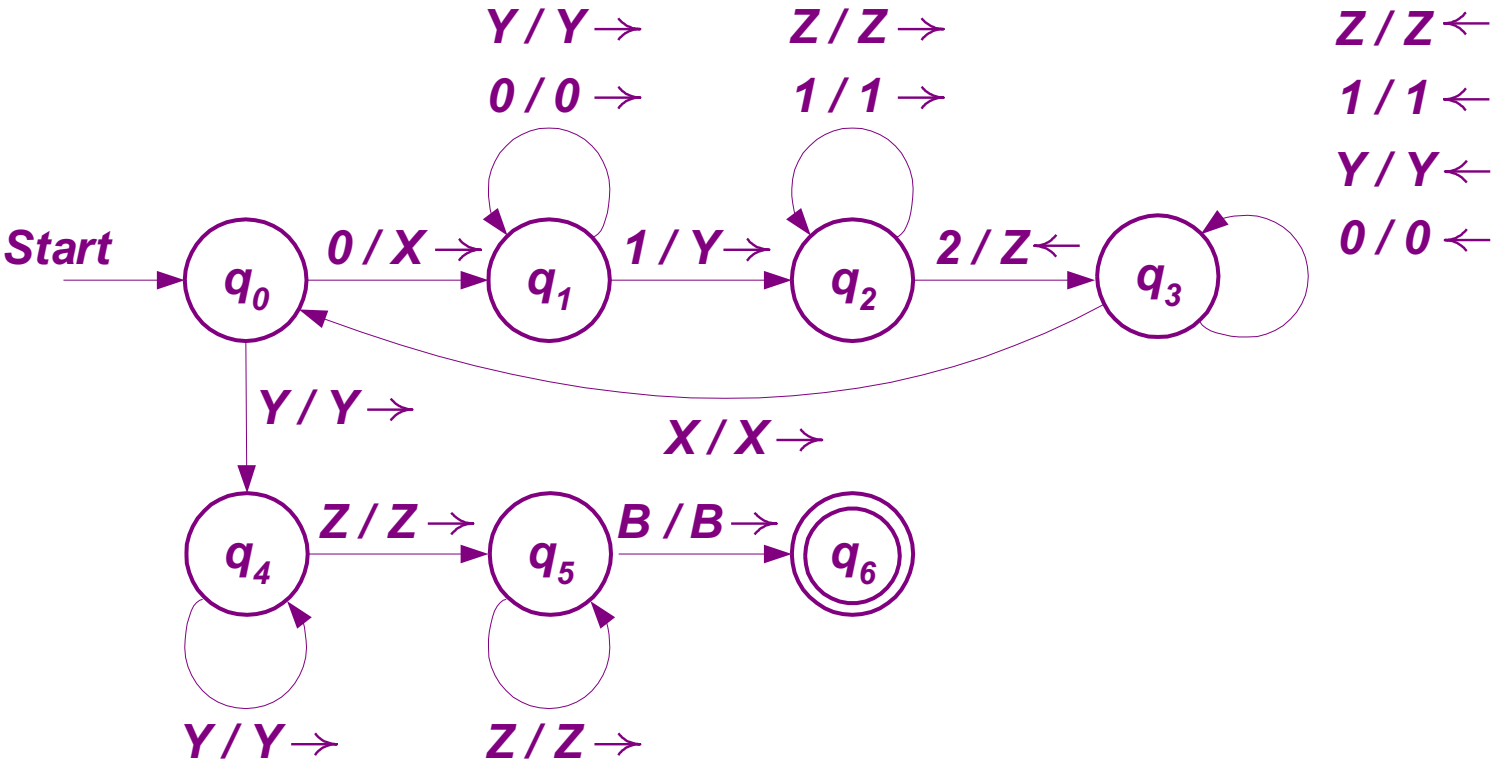
◇ 举例（续前例）



$q_0 001122 \vdash_M Xq_1 01122 \vdash_M X0q_1 1122 \vdash_M X0Yq_2 122 \vdash_M X0Y1q_2 22$
 $\vdash_M X0Yq_3 1Z2 \vdash_M^* q_3 X0Y1Z2 \vdash_M Xq_0 0Y1Z2 \vdash_M^* XXYYZq_2 2$
 $\vdash_M XXYYq_3 ZZ \vdash_M^* Xq_3 XYZZ \vdash_M XXq_0 YYZZ \vdash_M^* XXYYq_4 ZZ$
 $\vdash_M XXYYZq_5 Z \vdash_M XXYYZZq_5 B \vdash_M XXYYZZBq_6 B$

若输入00112，该图灵机会在哪个状态停机

- A q0
- B q1
- C q2
- D q3

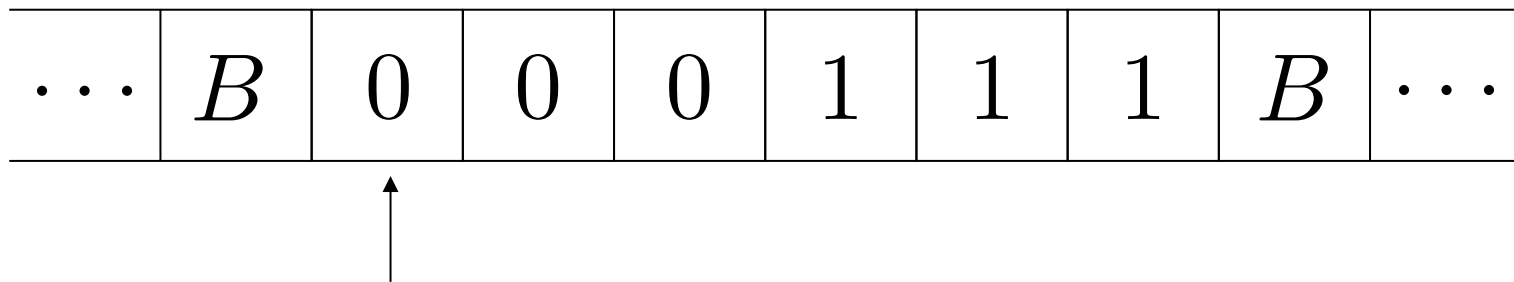


提交

设计图灵机

例1：设计图灵机，接受语言 $\{0^n 1^n | n \geq 1\}$ 。

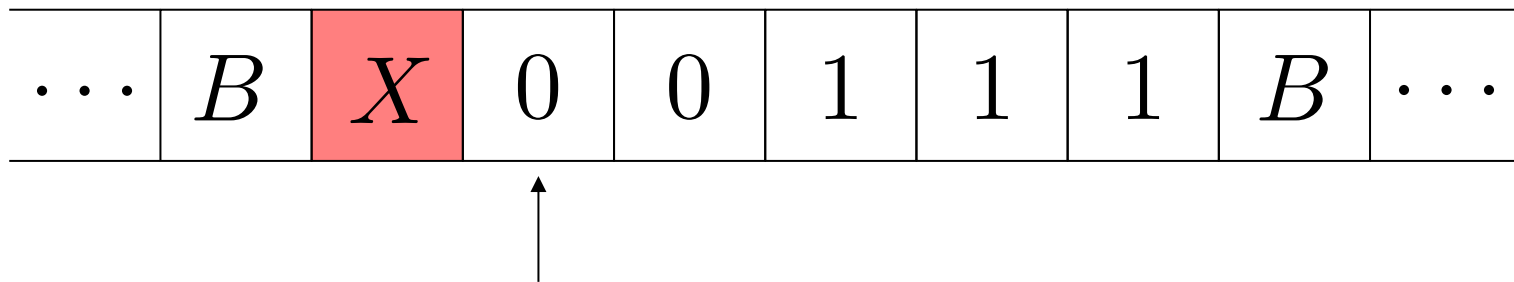
解：基本思想就是匹配，最左边的0和最左边的1逐个匹配，匹配完了，移动到末尾结束。在这个过程中，当找到最左边的1后，会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

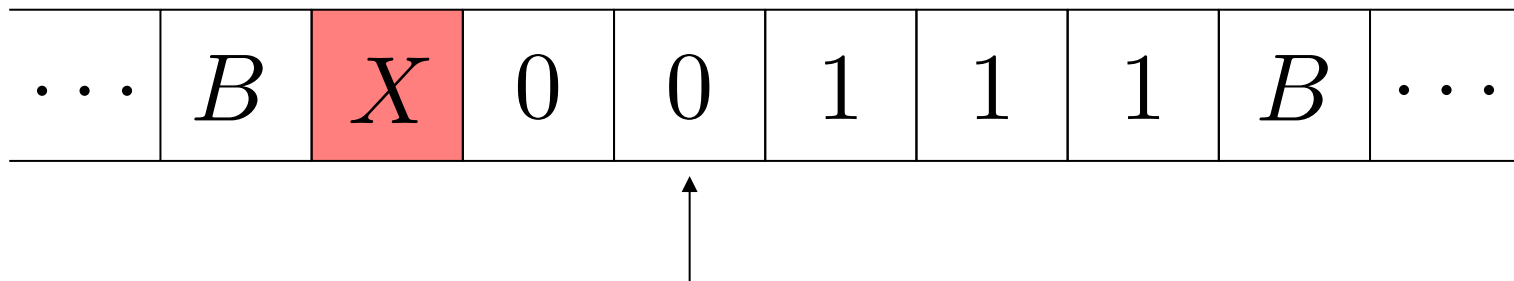
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

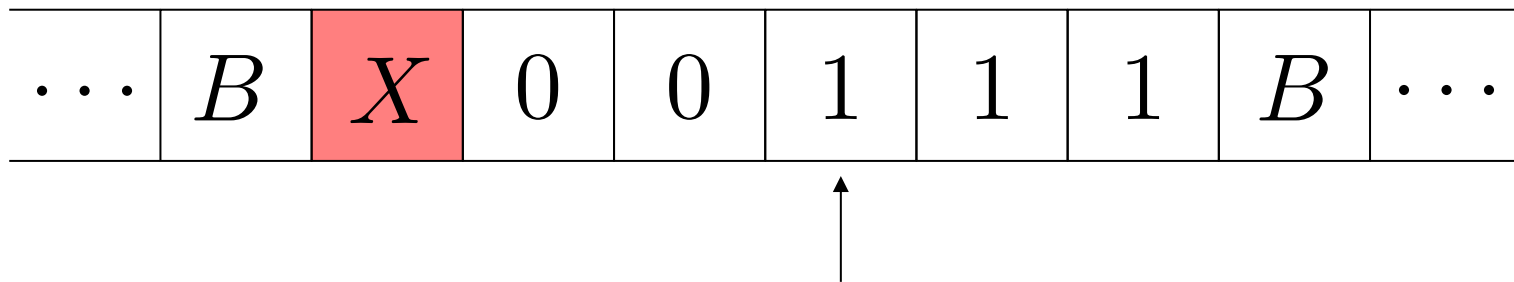
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1：设计图灵机，接受语言 $\{0^n 1^n | n \geq 1\}$ 。

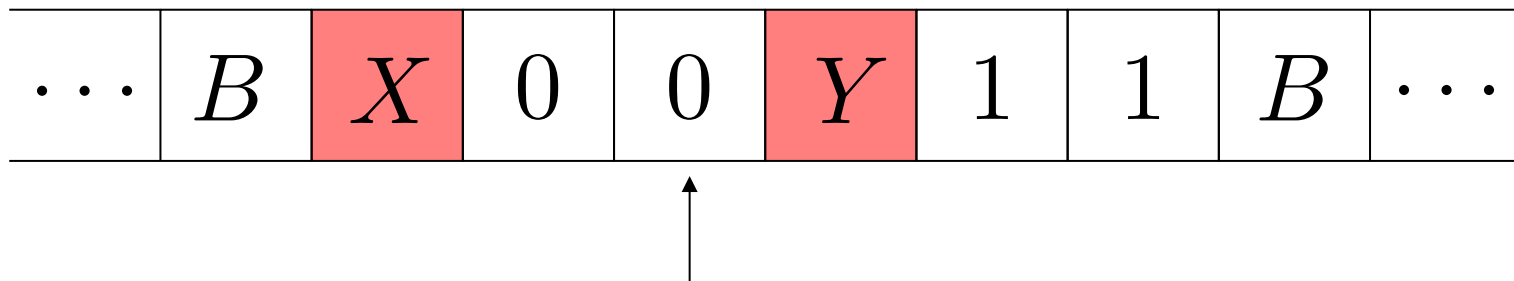
解：基本思想就是匹配，最左边的0和最左边的1逐个匹配，匹配完了，移动到末尾结束。在这个过程中，当找到最左边的1后，会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

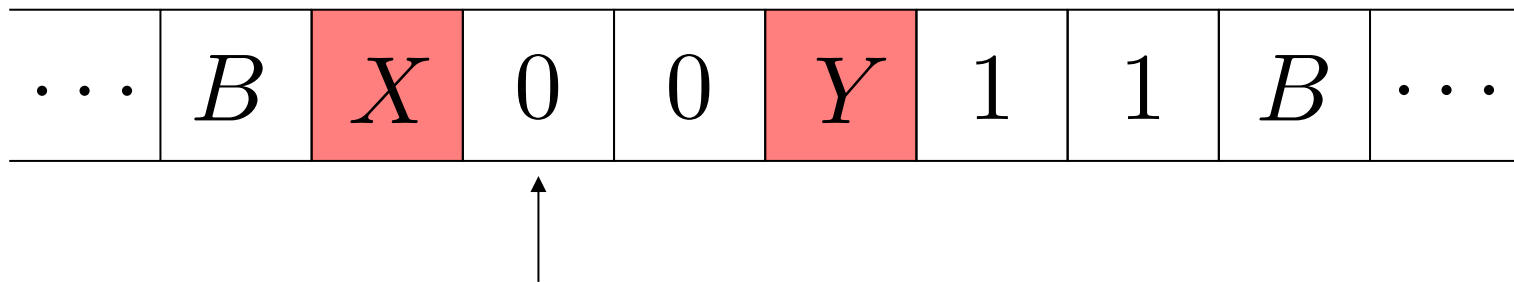
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1：设计图灵机，接受语言 $\{0^n 1^n | n \geq 1\}$ 。

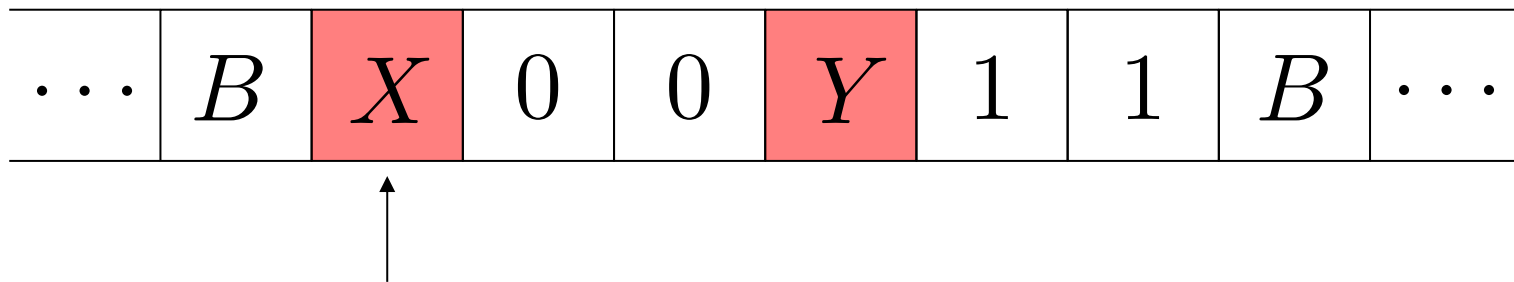
解：基本思想就是匹配，最左边的0和最左边的1逐个匹配，匹配完了，移动到末尾结束。在这个过程中，当找到最左边的1后，会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

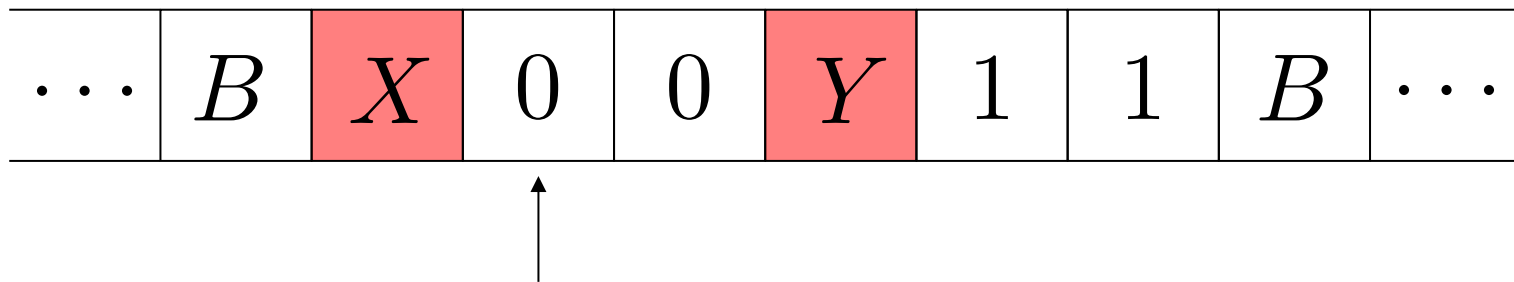
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1：设计图灵机，接受语言 $\{0^n 1^n | n \geq 1\}$ 。

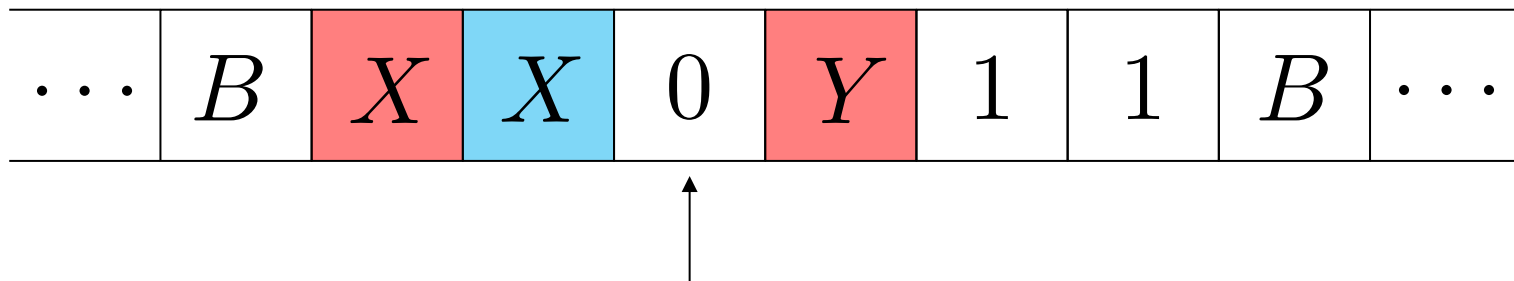
解：基本思想就是匹配，最左边的0和最左边的1逐个匹配，匹配完了，移动到末尾结束。在这个过程中，当找到最左边的1后，会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

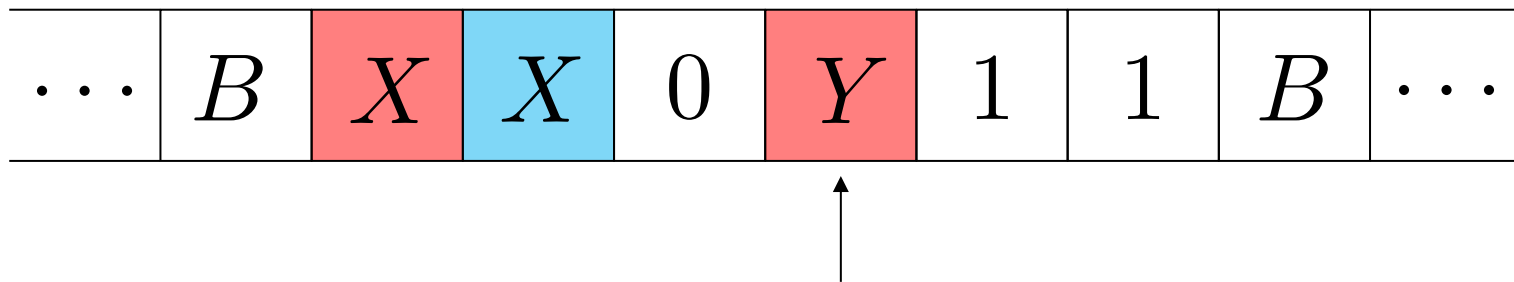
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

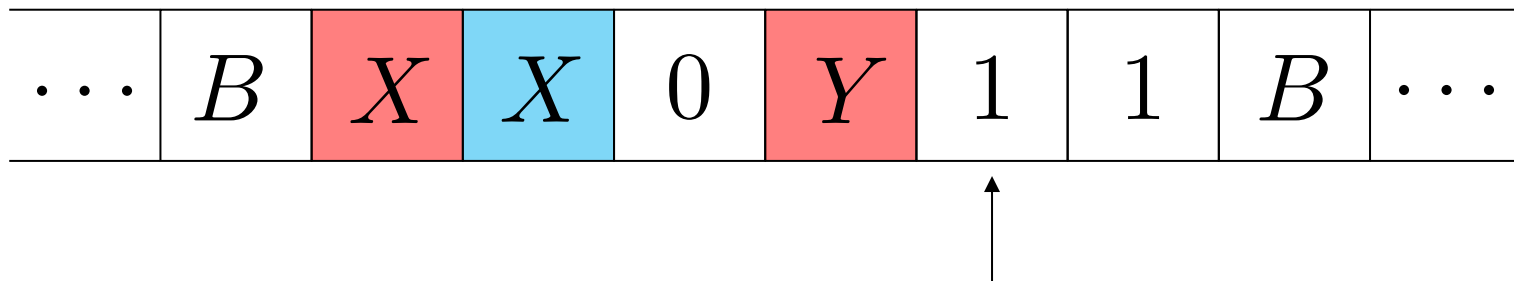
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

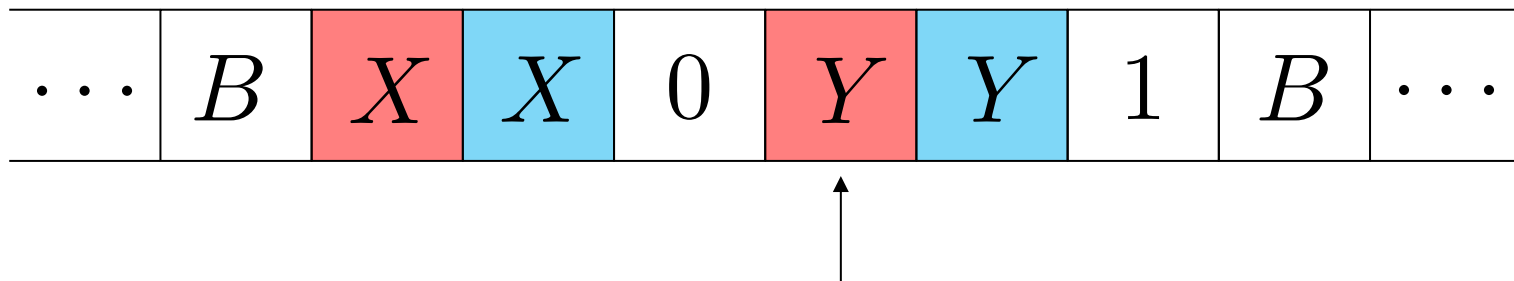
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

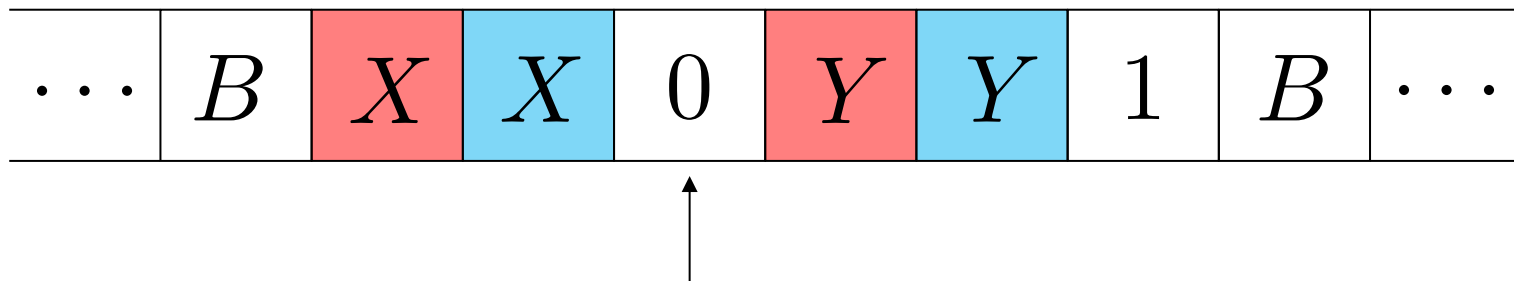
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

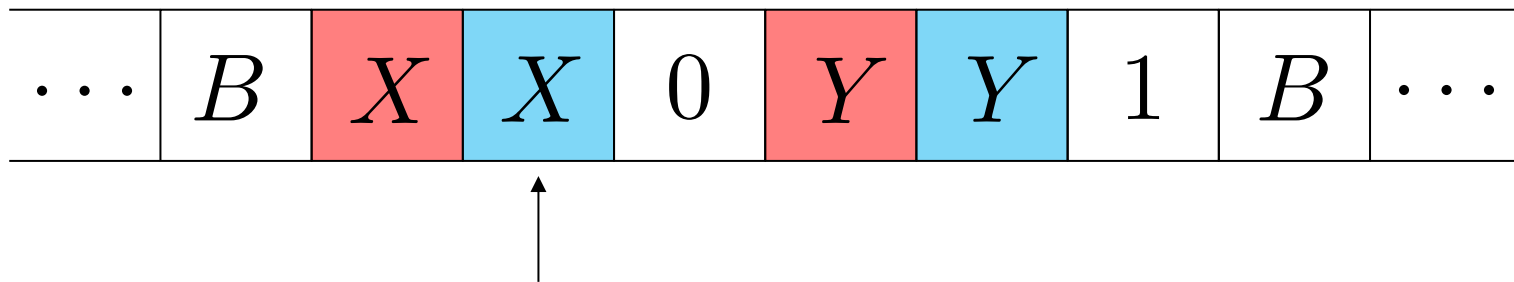
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

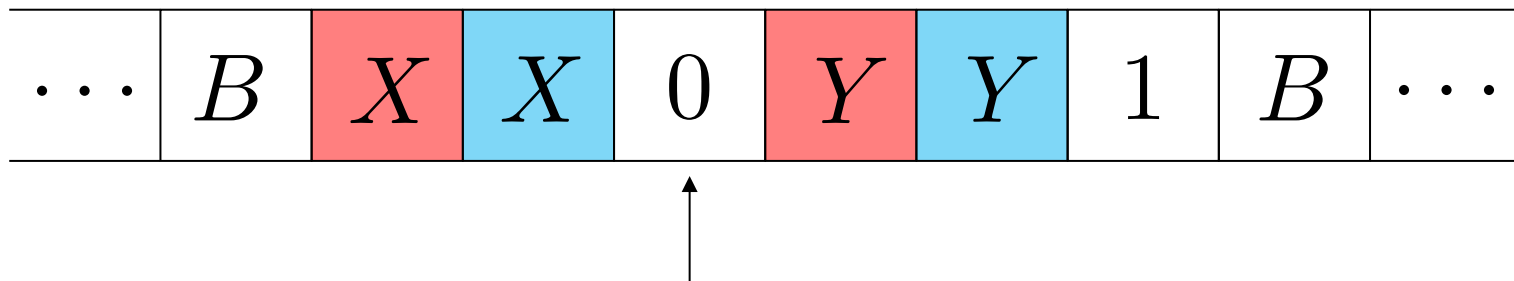
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

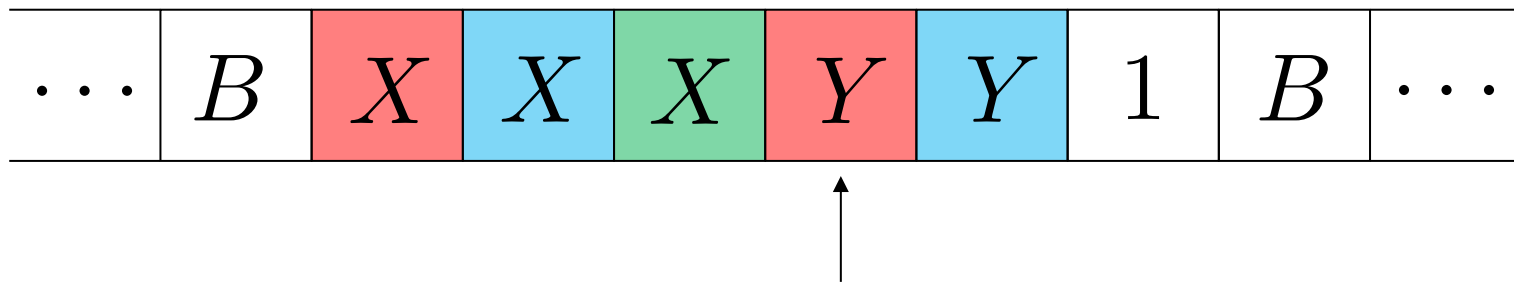
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

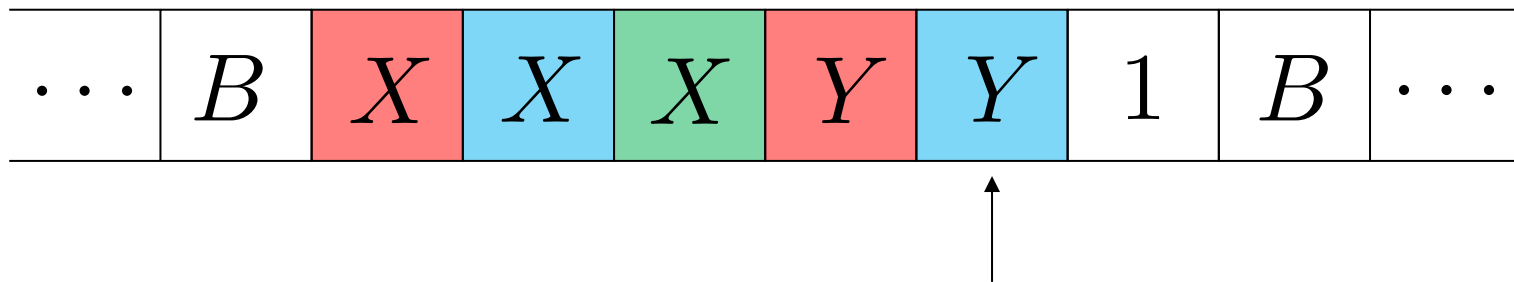
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

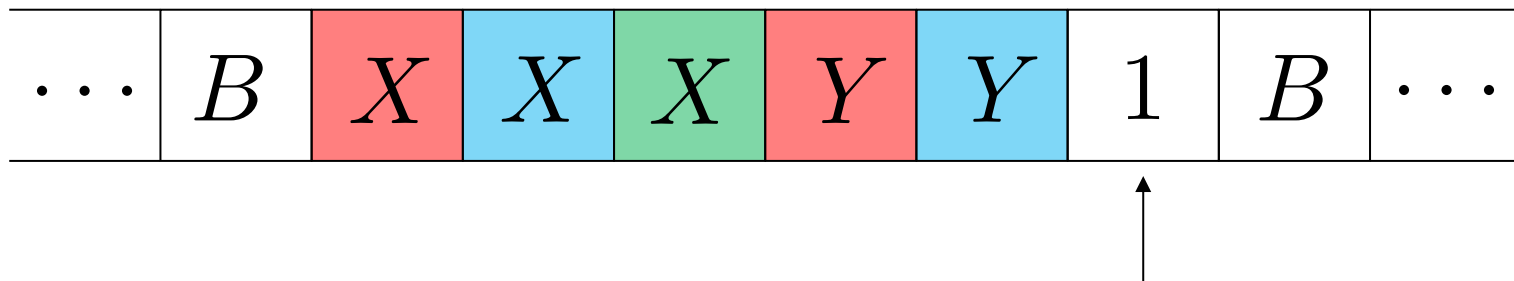
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

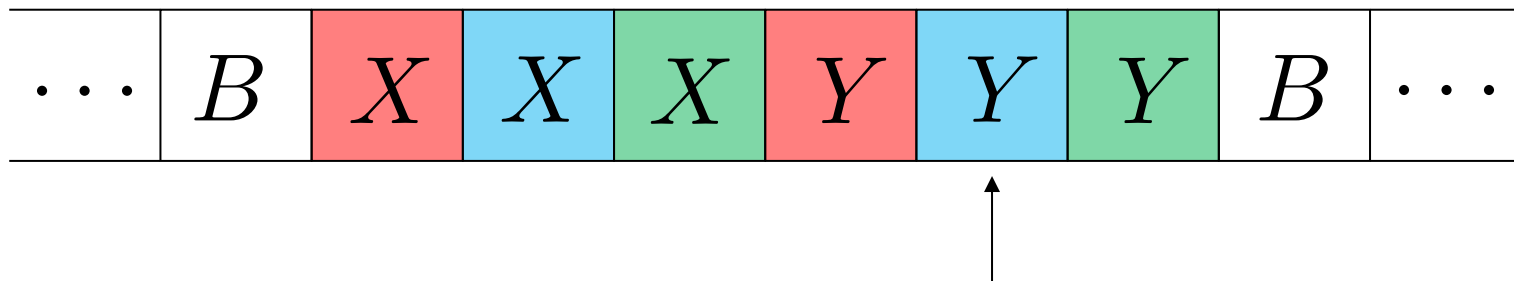
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1：设计图灵机，接受语言 $\{0^n 1^n | n \geq 1\}$ 。

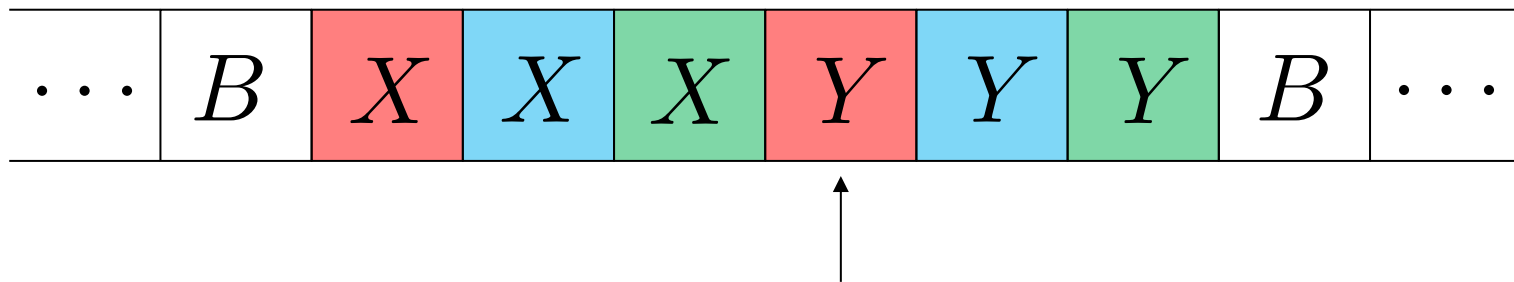
解：基本思想就是匹配，最左边的0和最左边的1逐个匹配，匹配完了，移动到末尾结束。在这个过程中，当找到最左边的1后，会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1：设计图灵机，接受语言 $\{0^n 1^n | n \geq 1\}$ 。

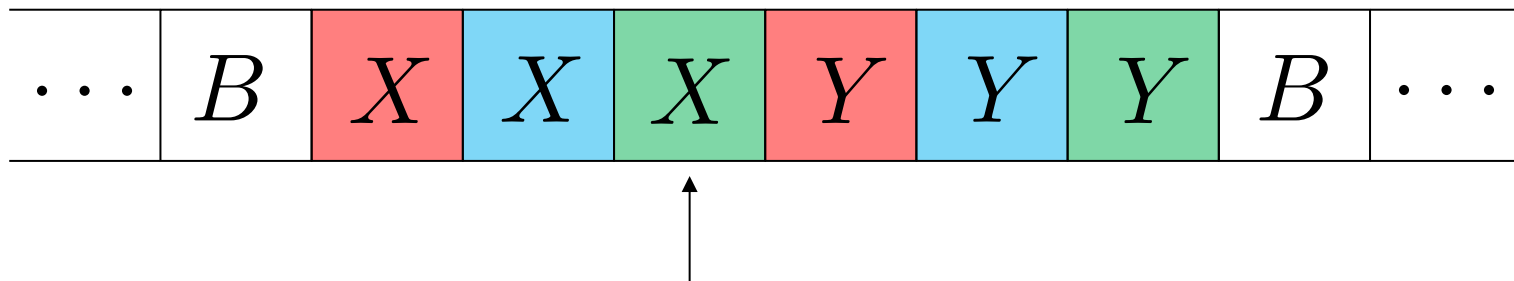
解：基本思想就是匹配，最左边的0和最左边的1逐个匹配，匹配完了，移动到末尾结束。在这个过程中，当找到最左边的1后，会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1：设计图灵机，接受语言 $\{0^n 1^n | n \geq 1\}$ 。

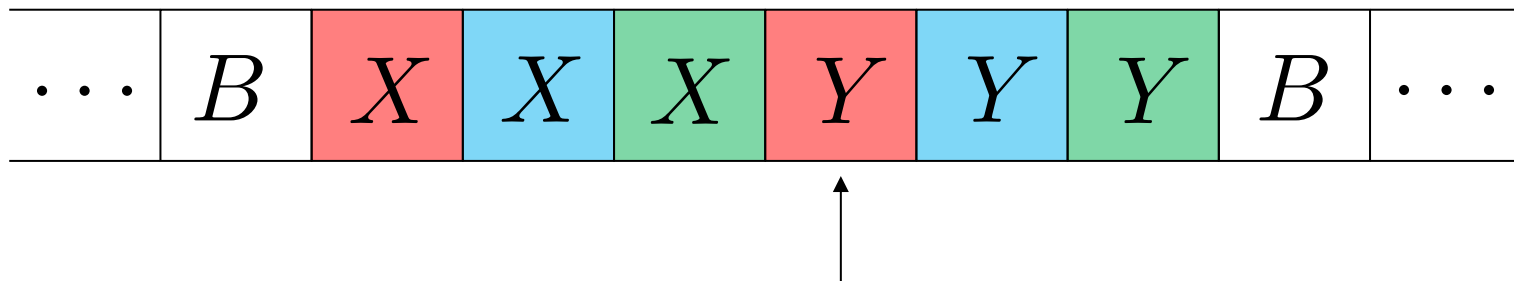
解：基本思想就是匹配，最左边的0和最左边的1逐个匹配，匹配完了，移动到末尾结束。在这个过程中，当找到最左边的1后，会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

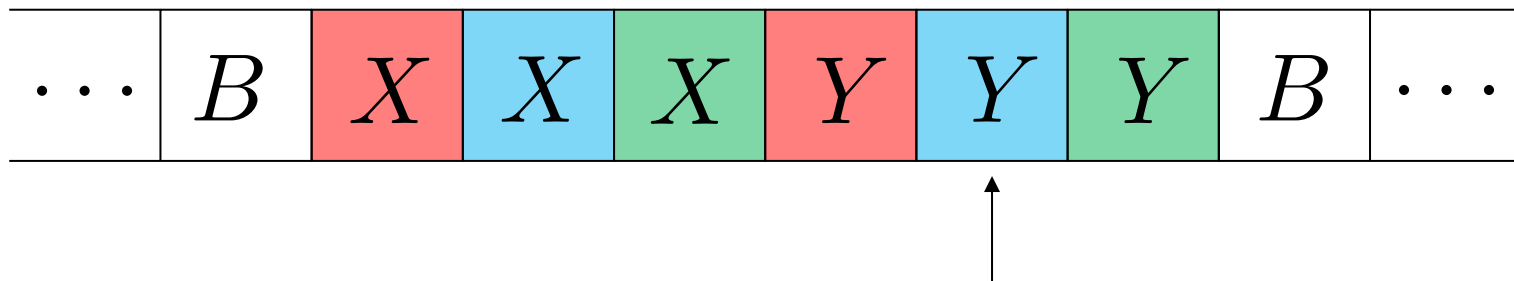
解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1：设计图灵机，接受语言 $\{0^n 1^n | n \geq 1\}$ 。

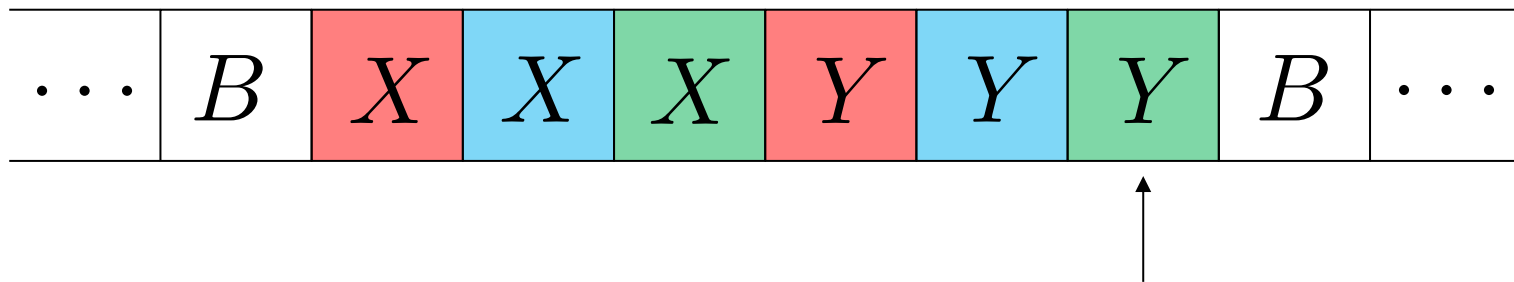
解：基本思想就是匹配，最左边的0和最左边的1逐个匹配，匹配完了，移动到末尾结束。在这个过程中，当找到最左边的1后，会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

例1：设计图灵机，接受语言 $\{0^n 1^n | n \geq 1\}$ 。

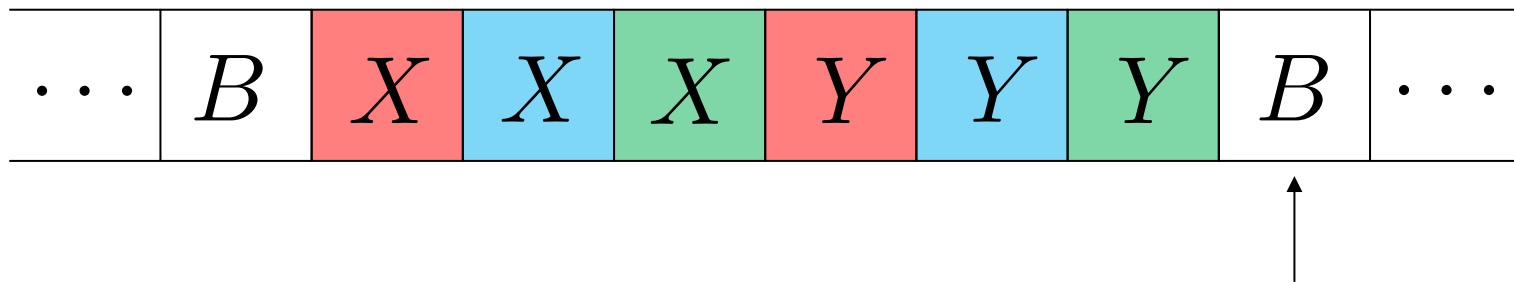
解：基本思想就是匹配，最左边的0和最左边的1逐个匹配，匹配完了，移动到末尾结束。在这个过程中，当找到最左边的1后，会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

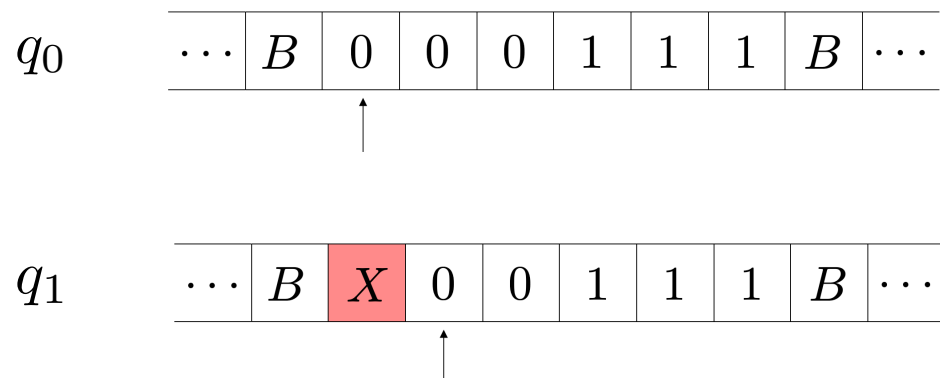
例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



设计图灵机

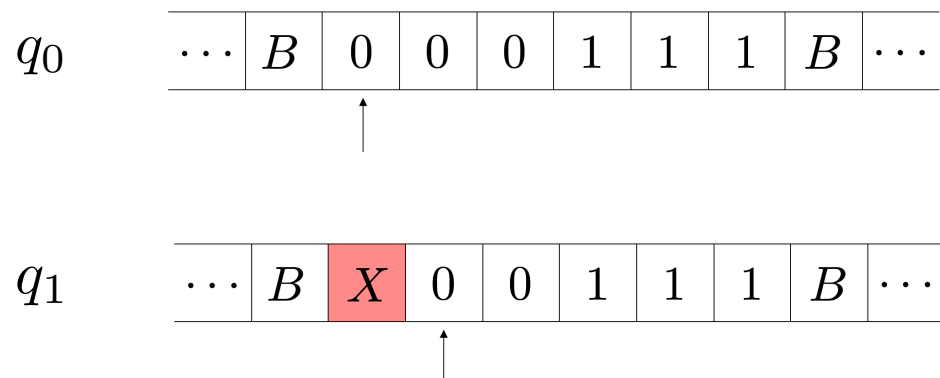
例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。



q_0 : 初始状态。在此状态下, X 和 Y 的数量是一样的, 或者叫“匹配”。当 M 每次回到剩下最左边的 0 时, M 也进入 0 。遇见最左的 0 , 将其改写为 X 。(生成 X)

设计图灵机

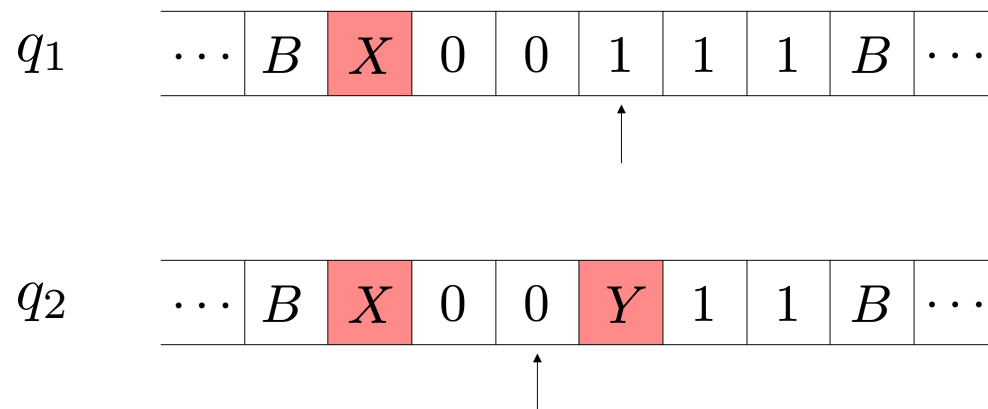
例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。



q_1 : 不停向右略过所有发现的0和Y, 直到碰到第一个1 (最左边的1)。碰到后, 将其改写为Y, 然后进入 q_2 , 向左移动。(生成Y)

设计图灵机

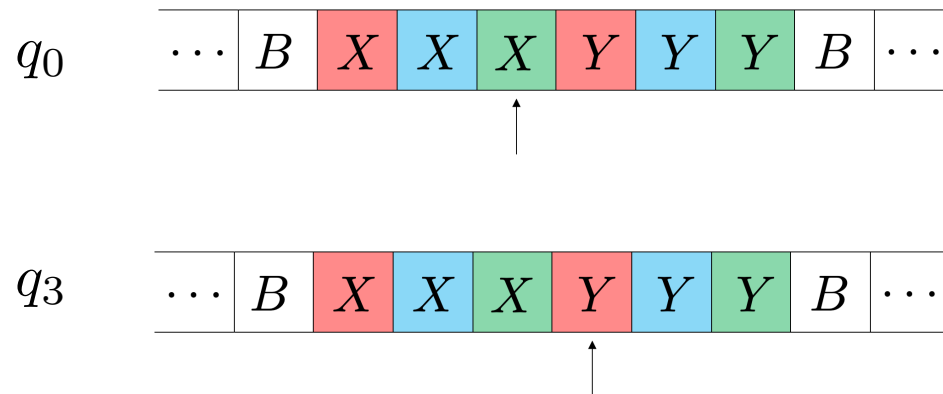
例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。



q_2 : 不停向左略过所有的0和Y, 直至碰到最右边的X (注意: 再往左的一定都是X, 如果有0的话, 一定在右边), 这时候进入状态 q_0 。

设计图灵机

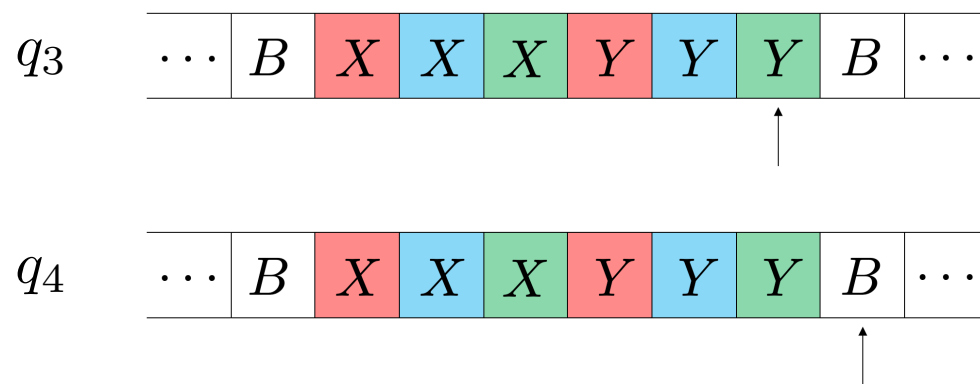
例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。



q_3 : 所有0和1的匹配完成 (q_0 状态遇见 Y 而不是0, 说明已经匹配完了), 不断向右直至末尾。

设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。



q_4 : 接受状态。

设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。

图灵机定义如下:

$$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, B\}, \delta, q_0, B, \{q_4\})$$

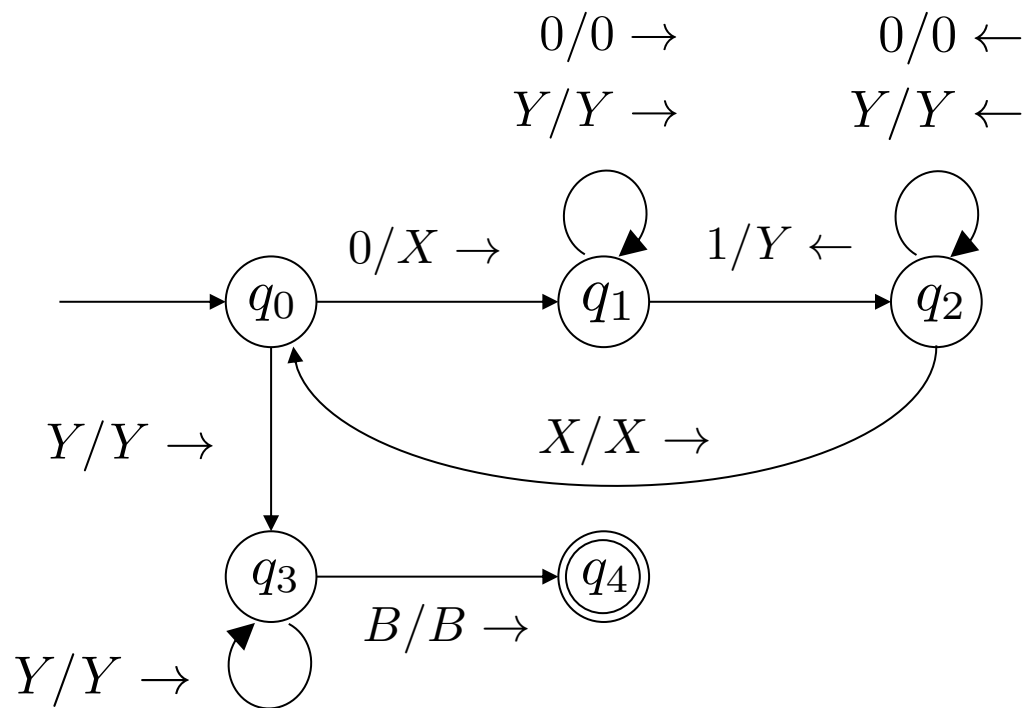
其中, 状态转移函数定义如下:

状态	0	1	X	Y	B
$\rightarrow q_0$	(q_1, X, R)	-	-	(q_3, Y, R)	-
q_1	$(q_1, 0, R)$	(q_2, Y, L)	-	(q_1, Y, R)	-
q_2	$(q_2, 0, L)$	-	(q_0, X, R)	(q_2, Y, L)	-
q_3	-	-	-	(q_3, Y, R)	(q_4, B, R)
$*q_4$	-	-	-	-	-

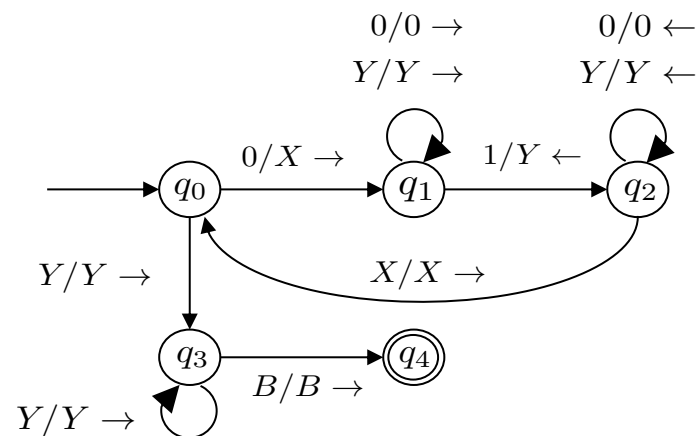
设计图灵机

例1: 设计图灵机, 接受语言 $\{0^n 1^n | n \geq 1\}$ 。

解: 基本思想就是匹配, 最左边的0和最左边的1逐个匹配, 匹配完了, 移动到末尾结束。在这个过程中, 当找到最左边的1后, 会回退回来再找最左边的0。这是因为0一定在1的左边。



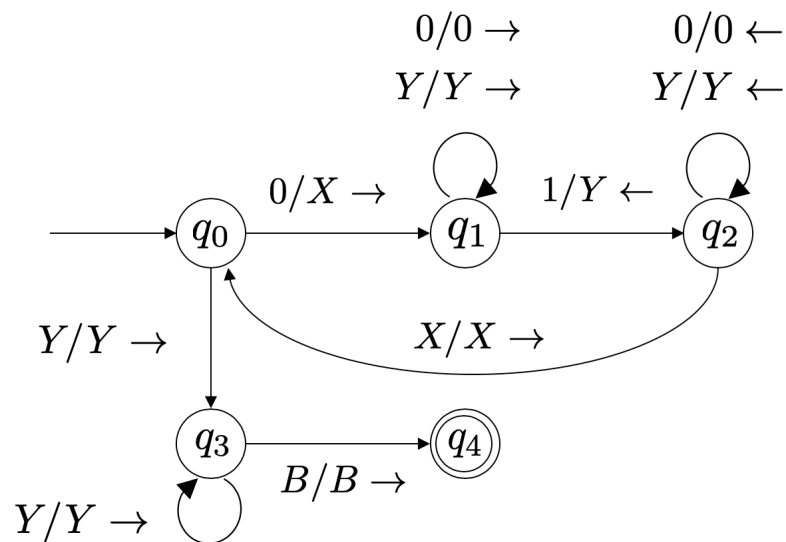
设计图灵机



下面来尝试解释一下每个状态的含义：

1. q_0 : 初始状态。在此状态下， X 和 Y 的数量是一样的，或者叫“匹配”。当 M 每次回到剩下最左边的0时， M 也进入0。遇见最左的0，将其改写为 X 。（生成 X ）
2. q_1 : 不停向右略过所有发现的0和 Y ，直到碰到第一个1（最左边的1）。碰到后，将其改写为 Y ，然后进入 q_2 ，向左移动。（生成 Y ）
3. q_2 : 不停向左略过所有的0和 Y ，直至碰到最右边的 X （注意：再往左的一定都是 X ，如果有0的话，一定在右边），这时候进入状态 q_0 。
4. q_3 : 所有0和1的匹配完成（ q_0 状态遇见 Y 而不是0，说明已经匹配完了），不断向右直至末尾。
5. q_4 : 接受状态。

设计图灵机

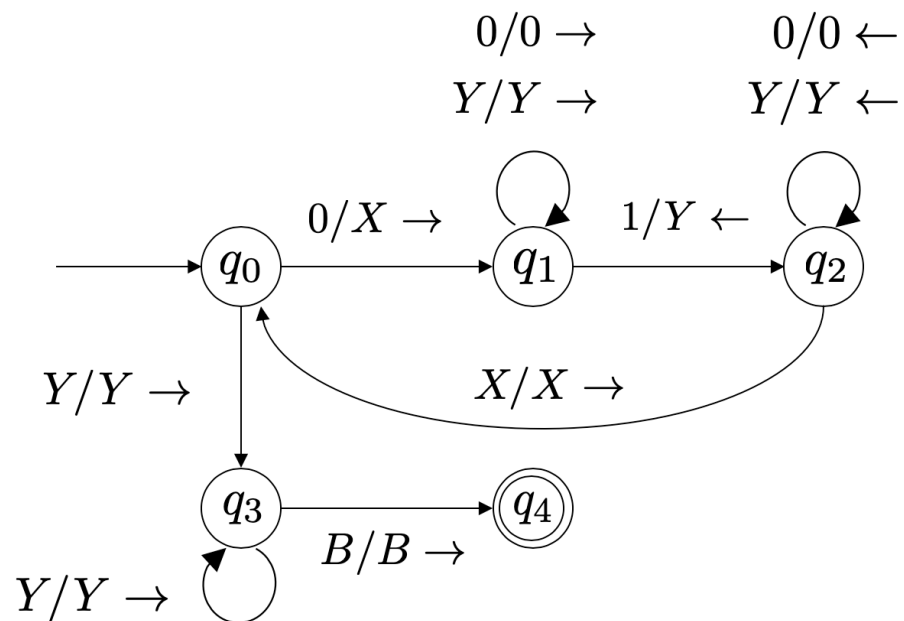


下面来分析一个输入0011的移动序列：

$$\begin{aligned}
 & q_0 0011 \vdash X q_1 011 \vdash X 0 q_1 11 \vdash X q_2 0Y1 \vdash q_2 X 0Y1 \vdash \\
 & X q_0 0Y1 \vdash X X q_1 Y1 \vdash X X Y q_1 1 \vdash X X q_2 Y Y \vdash X q_2 X Y Y \vdash \\
 & X X q_0 Y Y \vdash X X Y q_3 Y \vdash X X Y Y q_3 B \vdash X X Y Y B q_4 B
 \end{aligned}$$

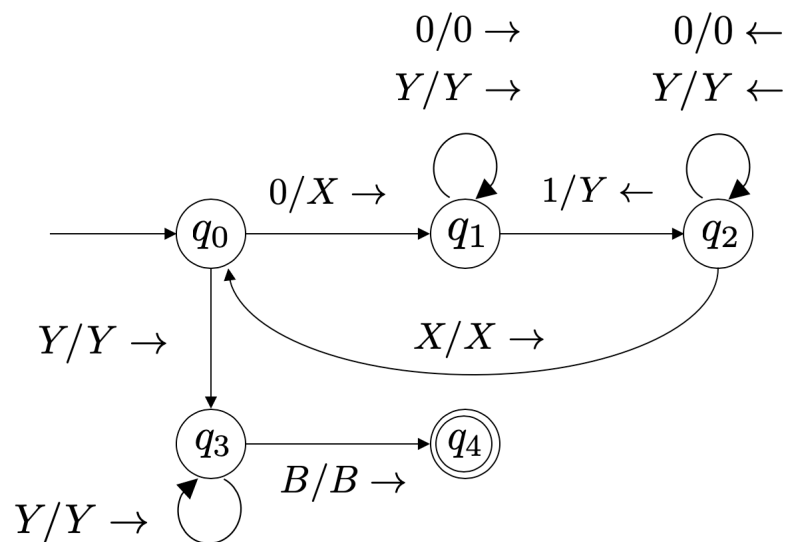
若输入0010，该图灵机会在哪个状态停机

- ☐ A q0
- ☒ B q1
- ☐ C q2
- ☐ D q3



提交

设计图灵机



那么再看一个反例0010。

$$\begin{aligned}
 & q_0 0010 \vdash X q_1 010 \vdash X 0 q_1 10 \vdash X q_2 0Y0 \vdash q_2 X 0Y0 \vdash \\
 & X q_0 0Y0 \vdash X X q_1 Y0 \vdash X X Y q_1 0 \vdash X X Y 0 q_1 B
 \end{aligned}$$

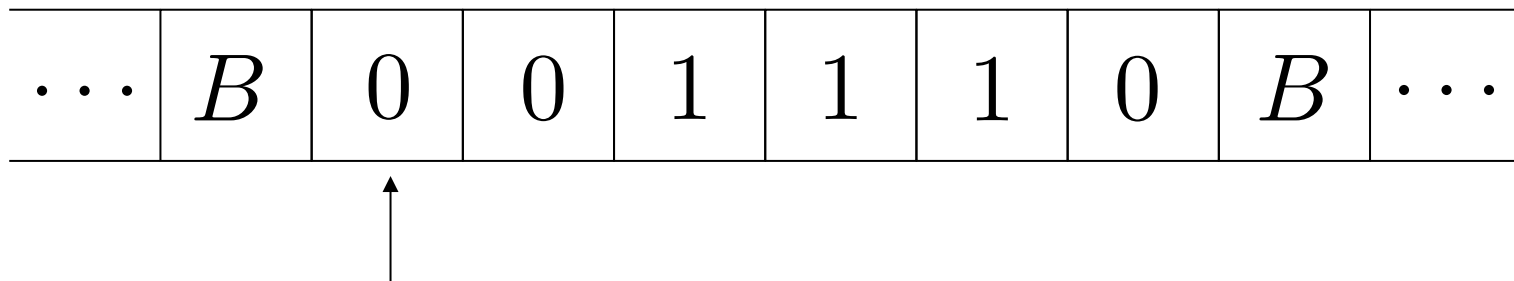
设计图灵机

例2: 为下列语言设计图灵机: 带有相同个数的0和1的串的集合。

设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

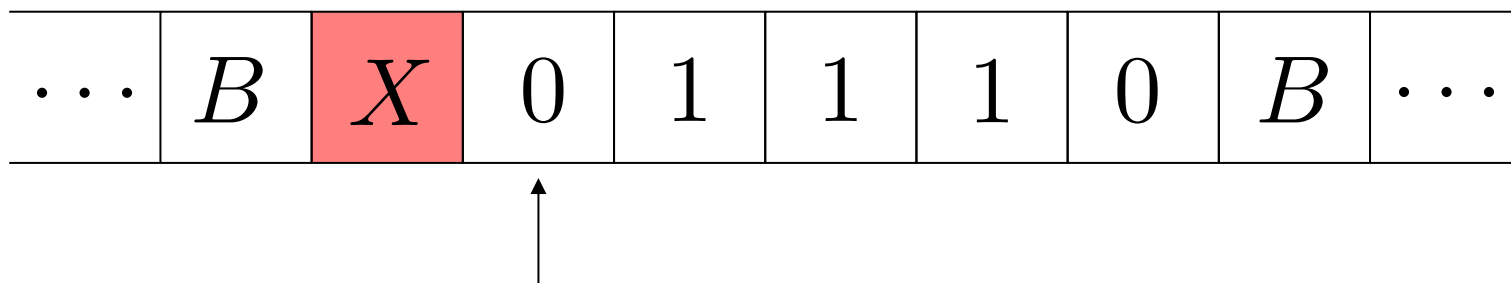
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

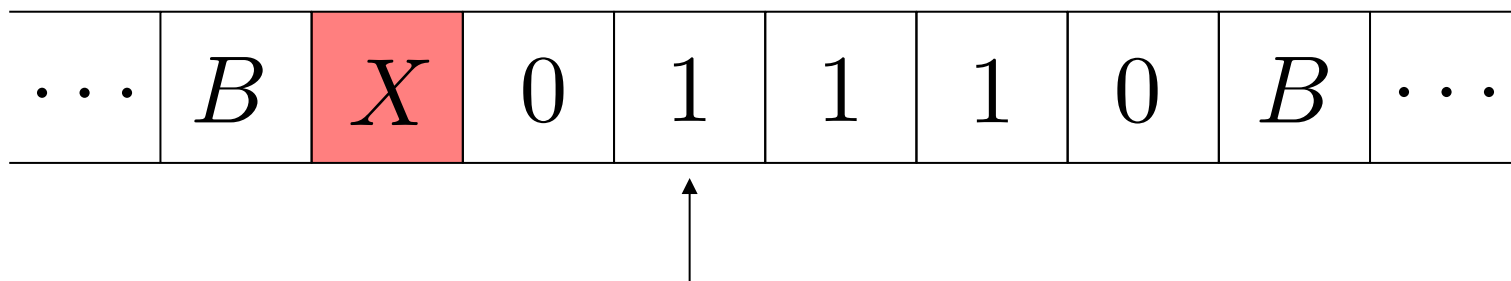
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

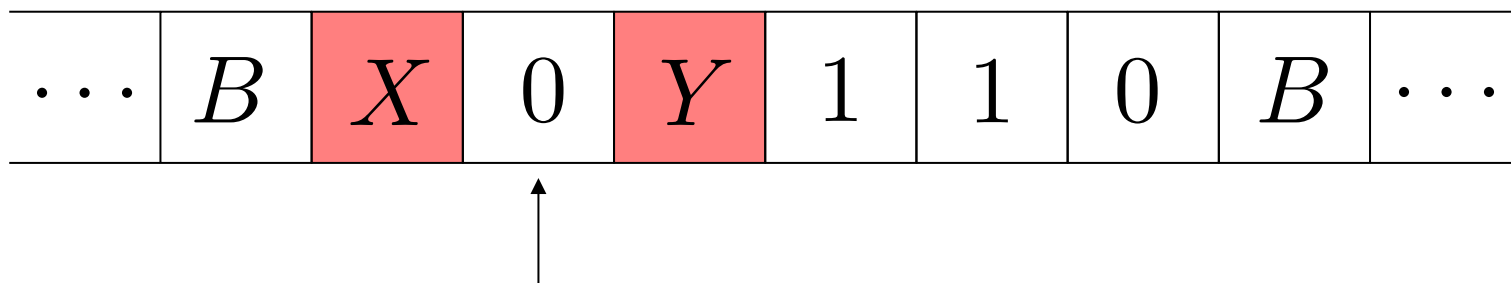
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

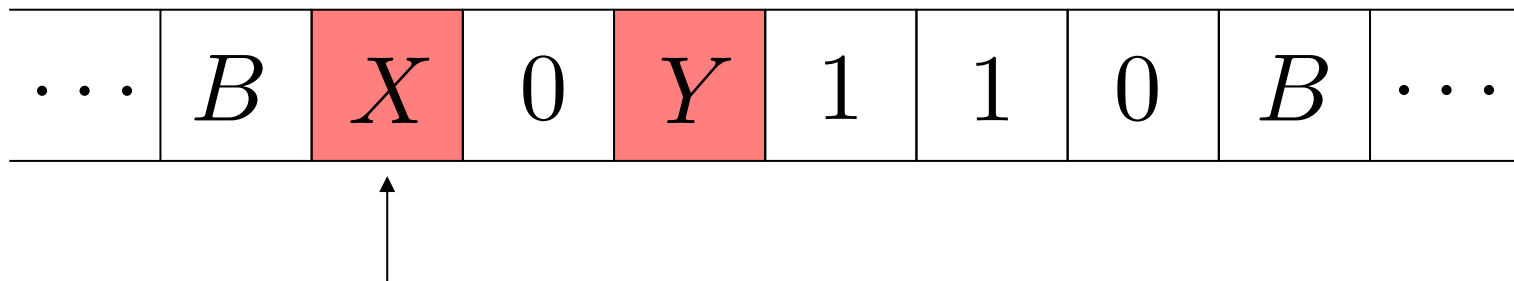
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

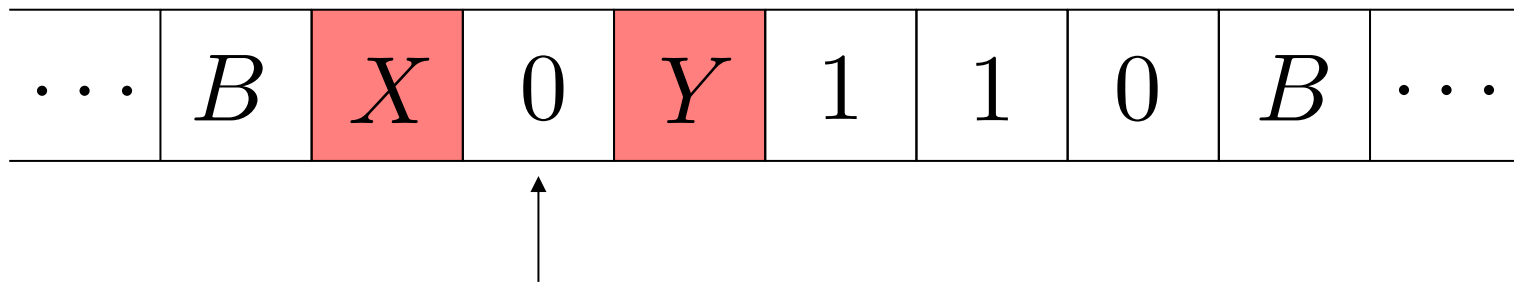
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

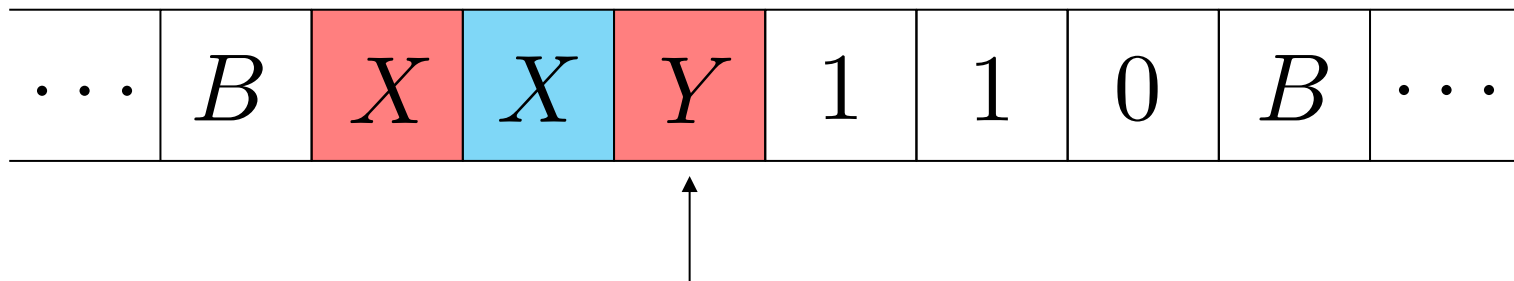
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

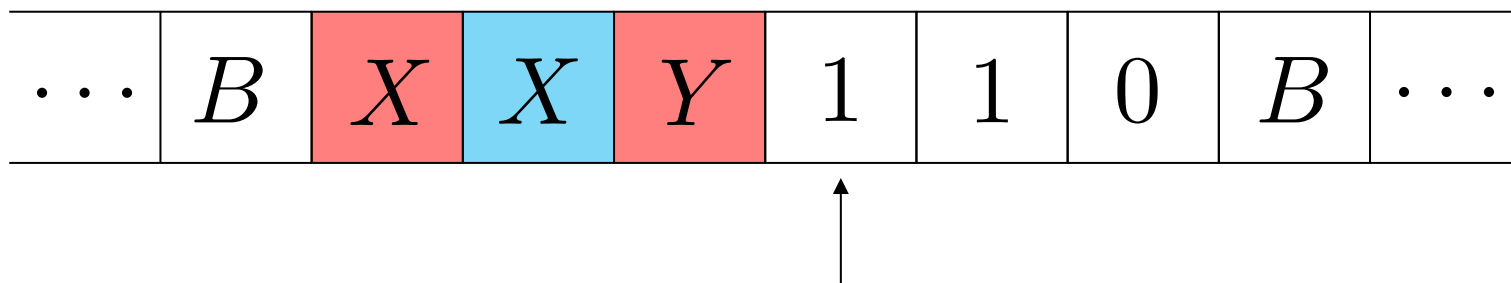
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

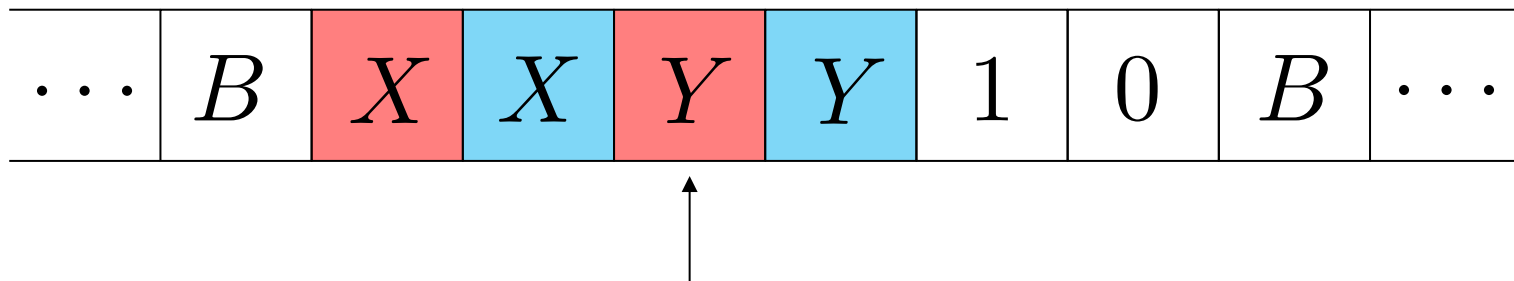
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

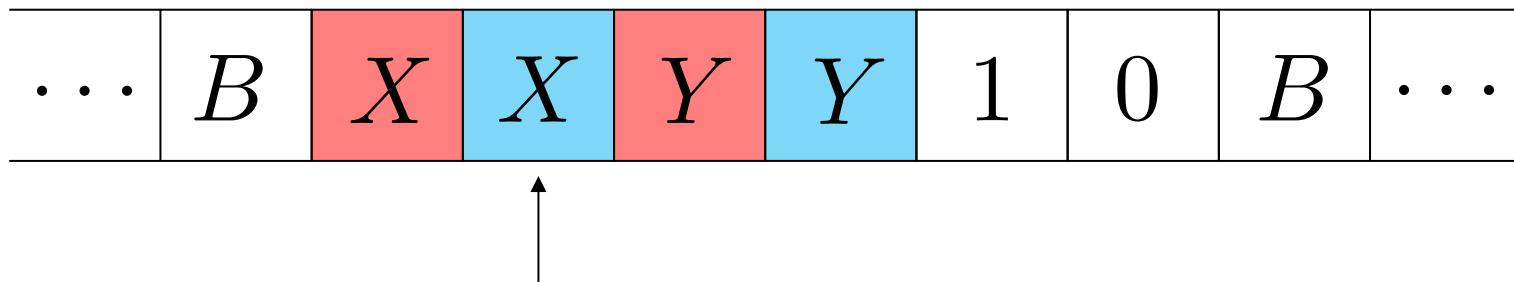
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

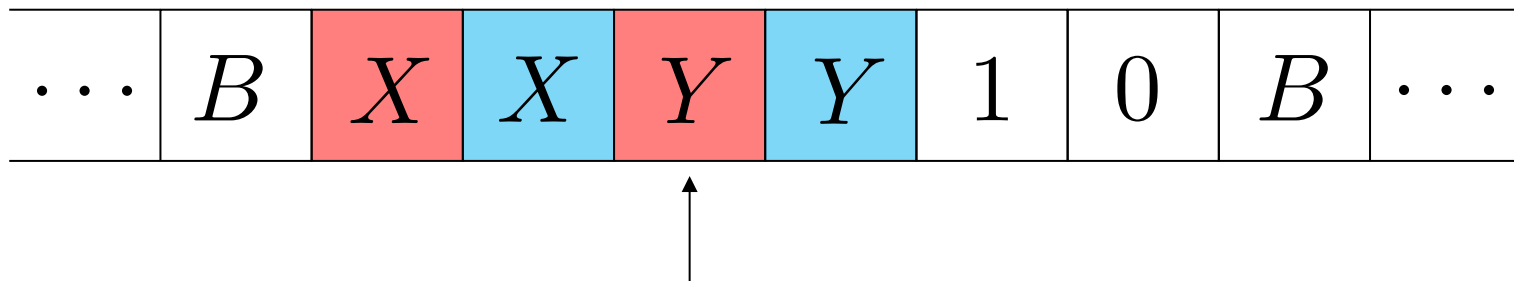
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

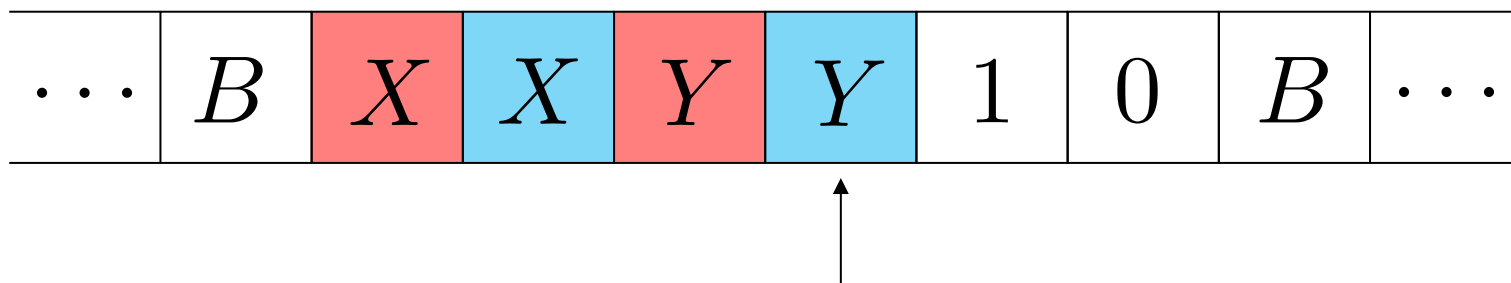
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

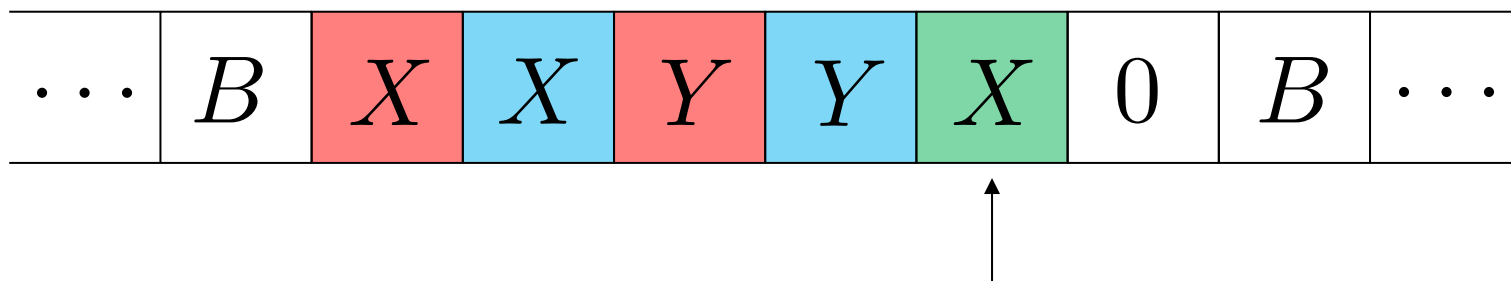
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

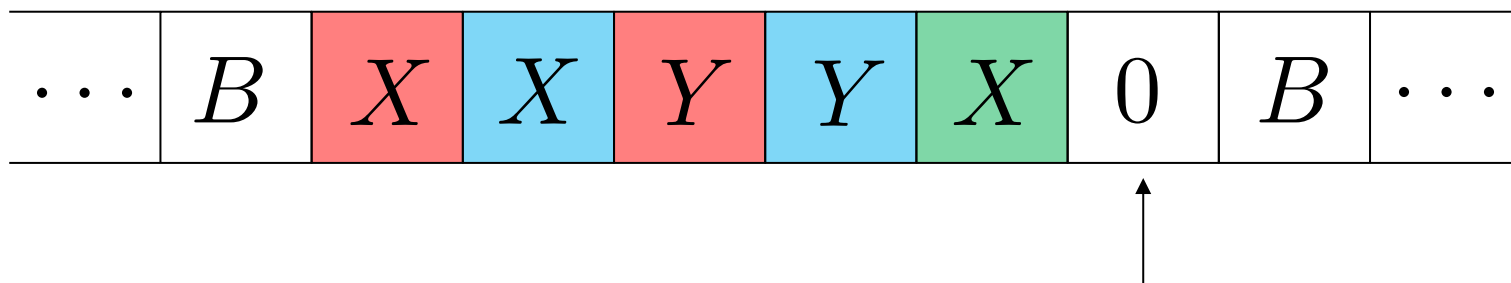
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

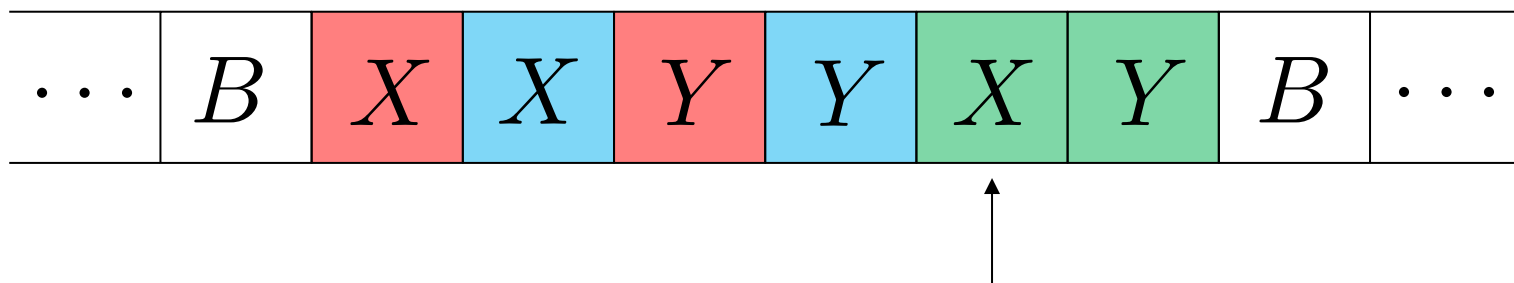
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。

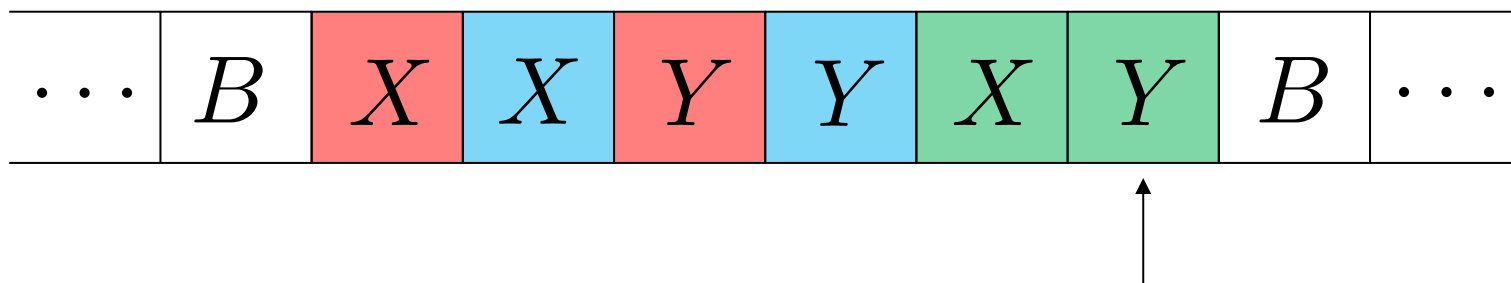


注意：X是指每次扫描匹配时的第一个元素，Y是指与之对应的第二个元素。
如果 $X = 1$ ，则 $Y = 0$ 。反之，如果 $X = 0$ ，则 $Y = 1$ 。

设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

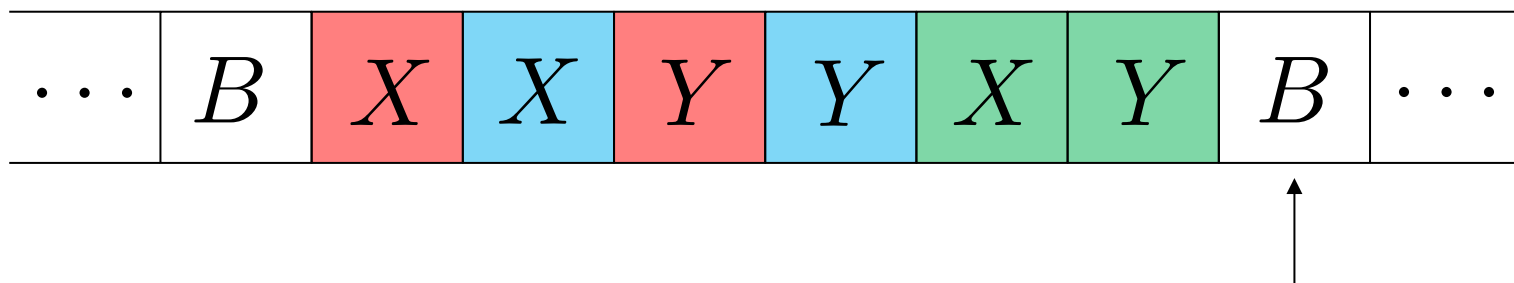
解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。

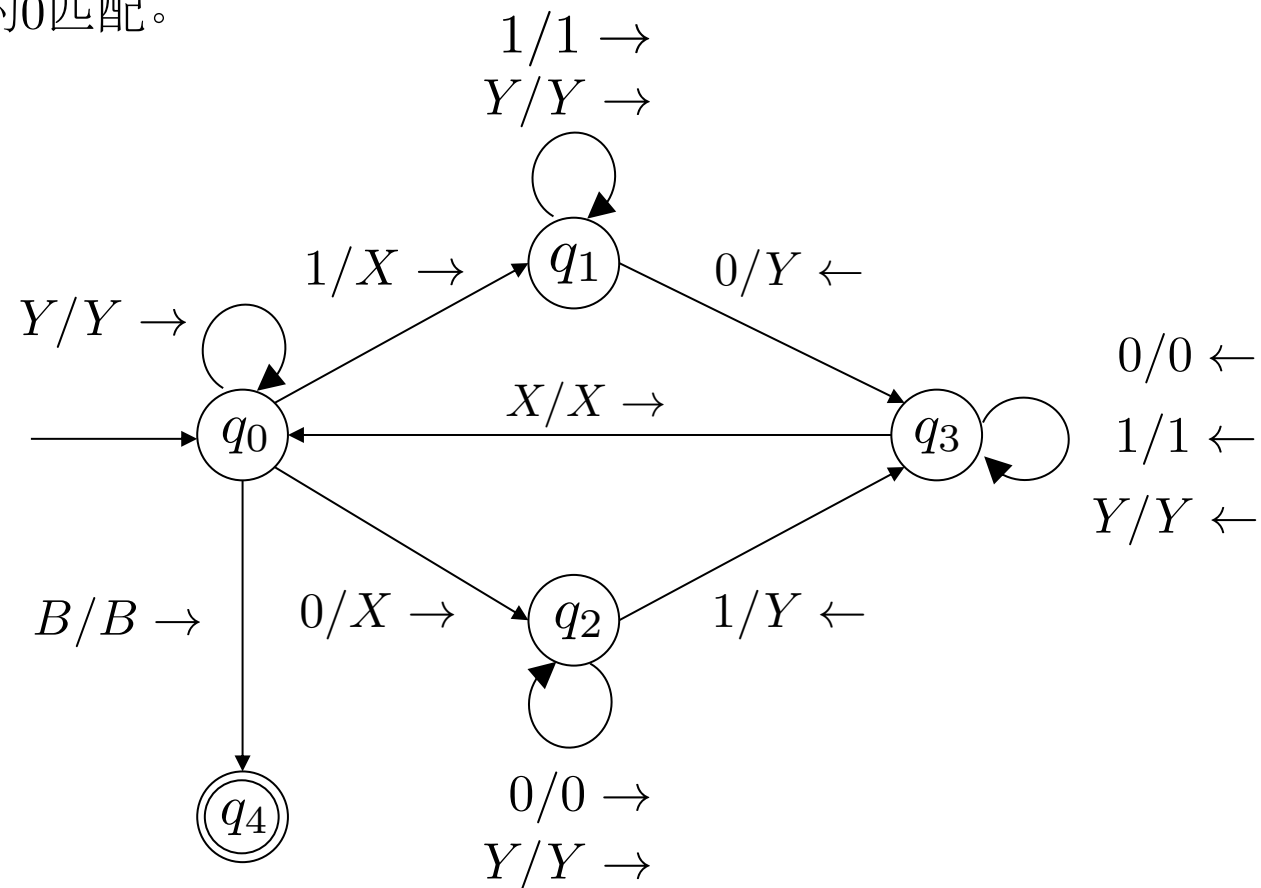
$$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, B\}, \delta, q_0, B, \{q_4\})$$

状态	0	1	X	Y	B
q_0	(q_2, X, R)	(q_1, X, R)	-	(q_0, Y, R)	(q_4, B, R)
q_1	(q_3, Y, L)	$(q_1, 1, R)$	-	(q_1, Y, R)	-
q_2	$(q_2, 0, R)$	(q_3, Y, L)	-	(q_2, Y, R)	-
q_3	$(q_3, 0, L)$	$(q_3, 1, L)$	(q_0, X, R)	(q_3, Y, L)	-
q_4	-	-	-	-	-

设计图灵机

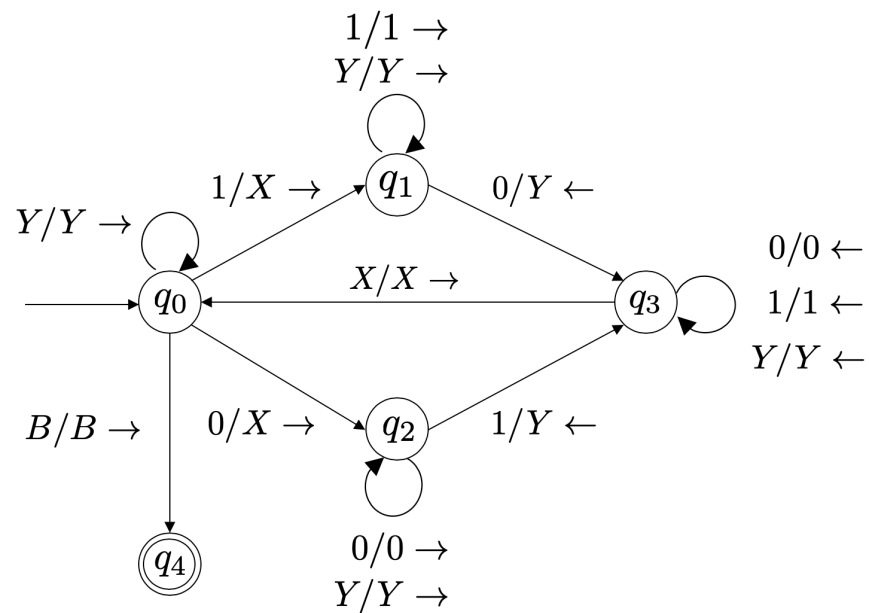
例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

解：基本思路还是匹配，分为两种情况：最左的0和最左的1匹配，或者最左的1和最左的0匹配。



设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。

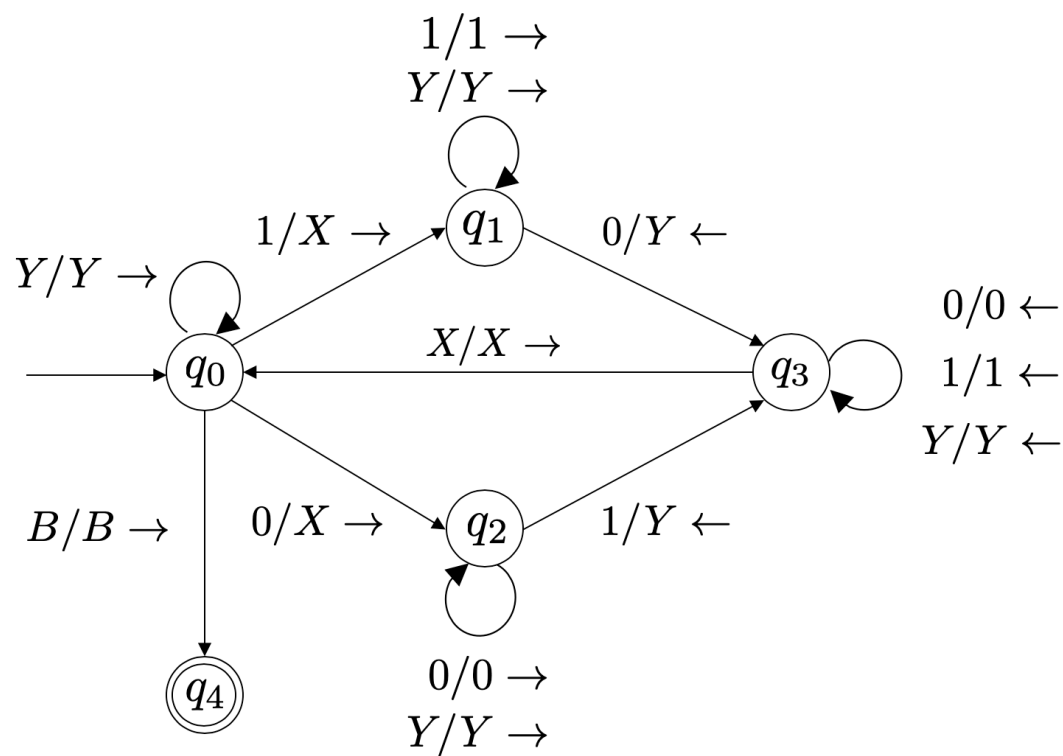


分析下一个例子1001:

$q_0 1001 \vdash X q_1 001 \vdash q_3 XY01 \vdash X q_0 Y01 \vdash XY q_0 01 \vdash$
 $XY X q_2 1 \vdash XY q_3 XY \vdash XY X q_0 Y \vdash XY XY q_0 B \vdash$
 $XY XY B q_4 B$

[多选]当图灵机处于q0状态时，带头可能指向哪些带符？（输入串为0和1组成的串）

- ☒ A 0
- ☒ B 1
- ☐ C X
- ☒ D Y
- ☒ E B



提交

[多选]以下哪些串能被该图灵机接收

考虑图灵机

$$M = (\{q_0, q_1, q_2, q_f\}, \{0, 1\}, \{0, 1, B\}, \delta, q_0, B, \{q_f\})$$

δ 包含下列规则集合:

$$b) \delta(q_0, 0) = (q_0, B, R); \delta(q_0, 1) = (q_1, B, R); \delta(q_1, 1) = (q_1, B, R); \delta(q_1, B) = (q_f, B, R)。$$

- ☒ **A** 011
- ☒ **B** 01
- ☐ **C** 100
- ☒ **D** 000111
- ☐ **E** 0010
- ☒ **F** 1

提交

设计图灵机

例4 考虑图灵机

$$M = (\{q_0, q_1, q_2, q_f\}, \{0, 1\}, \{0, 1, B\}, \delta, q_0, B, \{q_f\})$$

非形式化但清楚地描述语言 $L(M)$ ，如果 δ 包含下列规则集合：

$$b) \delta(q_0, 0) = (q_0, B, R); \delta(q_0, 1) = (q_1, B, R); \delta(q_1, 1) = (q_1, B, R); \delta(q_1, B) = (q_f, B, R)。$$

请描述这个语言

设计图灵机

例4 考虑图灵机

$$M = (\{q_0, q_1, q_2, q_f\}, \{0, 1\}, \{0, 1, B\}, \delta, q_0, B, \{q_f\})$$

非形式化但清楚地描述语言 $L(M)$ ，如果 δ 包含下列规则集合：

$$b) \delta(q_0, 0) = (q_0, B, R); \delta(q_0, 1) = (q_1, B, R); \delta(q_1, 1) = (q_1, B, R); \delta(q_1, B) = (q_f, B, R)。$$

$$L(M) = L(0^*11^*)$$

即其中的串可非形式地描述为具有如下特征的 0、1 串：至少包含一个“1”，且第一个“1”之前只可能出现“0”，每个“1”之后只可能跟随“1”。